FLASHATTENTION: A MATHEMATICIAN'S WALKTHROUGH

ERNEST YEUNG

Abstract. These notes develop the mathematics of the attention mechanism and its IO-efficient implementation (FlashAttention) from first principles, at the level of precision expected in pure mathematics. We maintain a strict separation between two levels of description: (1) the *computational level*, where scalars are elements of $\mathbb{Z}/2^n\mathbb{Z}$ (integer types, a ring) or of the finite set $\mathbb{F}_{p,e}$ (floating-point types, which satisfy no standard algebraic axioms); and (2) the *mathematical model level*, over the field $\mathbb{R}$, where the softmax map is a smooth surjection onto the interior of the standard simplex, attention is a smooth map between spaces of matrices, and the FlashAttention algorithm follows from an associativity identity for a running-maximum softmax accumulator. All smoothness and gradient claims are Level-2 statements. The audience is assumed to be comfortable with finite-dimensional linear algebra, basic smooth manifold theory, and the language of categories.

Contents

Date: 2026.

Key words and phrases. attention mechanism, softmax, tiled matrix multiplication, IO complexity, online softmax, GPU memory hierarchy.

Part 1. Foundations: Data Types, Scalar Domains, and Layers

1. The Scalar Domain

In pure mathematics the natural scalar domain for linear algebra is a *field*: a commutative ring in which every non-zero element has a multiplicative inverse. The reals $(\mathbb{R}, +, \cdot)$ are the prototypical complete ordered field.

In every practical implementation of the algorithms discussed in these notes, however, the scalar domain is a finite-precision floating-point type such as IEEE 754 `float32`, `float16`, or `bfloat16`. It is important to be explicit about what algebraic structure these types possess, because they do *not* satisfy the standard axioms.

Definition 1.1 (IEEE 754 floating-point set)**.** Let $\mathbb{F}_{p,e}$ denote the set of IEEE 754 floating-point numbers with p-bit mantissa and e-bit exponent. Concretely, each element is a rational number of the form

$$(-1)^s \cdot m \cdot 2^E, \qquad s \in \{0,1\},\ m \in \{0,\dots,2^p-1\},\ E \in \{E_{\min},\dots,E_{\max}\},$$

together with the special values $\pm\infty$ and NaN. The set $\mathbb{F}_{p,e}$ is *finite*: for `float32`, $|\mathbb{F}_{32}| = 2^{32}$ (counting all bit patterns).

Proposition 1.2 (Floating-point arithmetic is not a ring)**.** *The set $\mathbb{F}_{p,e}$ with IEEE 754 operations $\oplus, \otimes$ defined by*

$$a \oplus b \ := \ \mathrm{round}(a+b), \qquad a \otimes b \ := \ \mathrm{round}(a \cdot b),$$

(where round *is the nearest-even rounding function and $a+b$, $a\cdot b$ are exact real arithmetic) does* not *form a ring. In particular, $\oplus$ is not associative: there exist $a, b, c \in \mathbb{F}_{p,e}$ with $(a \oplus b) \oplus c \neq a \oplus (b \oplus c)$.*

Counterexample for `float32`. Take $a = 1$, $b = 10^7$, $c = -10^7$. Then $(a \oplus b) \oplus c = (10^7 + 1) \oplus (-10^7) = 0$ (the $+1$ is lost to rounding when added to 10^7 in 24-bit mantissa arithmetic), whereas $a \oplus (b \oplus c) = a \oplus 0 = 1 \neq 0$. □

Remark 1.3 (Integer types are rings)**.** In contrast, n-bit integer types with wrapping overflow are the ring $\mathbb{Z}/2^n\mathbb{Z}$, a commutative ring with identity. Addition and multiplication satisfy all ring axioms exactly (with no rounding). This is why integer arithmetic is algebraically cleaner than floating-point arithmetic.

Remark 1.4 (Two levels of description)**.** These notes maintain a strict separation between two levels of mathematical description.

Level 1 (what hardware computes): the scalar domain is one of two finite sets.

(a) *Integer arithmetic.* The scalars are elements of $\mathbb{Z}/2^n\mathbb{Z}$ (e.g. `int32` with wrapping overflow), a genuine commutative ring with identity; all ring axioms hold exactly with no rounding. The data container $(\mathbb{Z}/2^n\mathbb{Z})^{n_1\times\cdots\times n_k}$, whose elements are matrices with entries in $\mathbb{Z}/2^n\mathbb{Z}$, is a free module over this ring — an exact algebraic object. Note: $\mathbb{Z}/2^n\mathbb{Z}$ is *not* a field (zero divisors exist: $2^{n/2}\cdot 2^{n/2}=0$) and *not* an integral domain.

(b) *Floating-point arithmetic.* The scalars are elements of $\mathbb{F}_{p,e}$, a *finite set* whose operations $\oplus,\otimes$ satisfy no standard algebraic axioms (Proposition 1.2). The data container $\mathbb{F}_{p,e}^{n_1\times\cdots\times n_k}$, whose elements are matrices with entries in $\mathbb{F}_{p,e}$, is merely a finite set: it carries no module structure, no natural topology beyond the discrete topology, and no notion of differentiability. On a finite discrete set, every map is trivially continuous; the concept of *smooth map* is vacuous (there are no non-constant paths, hence no derivatives to define).

Level 2 (mathematical model): where theorems are stated. Theoretical analysis is conducted over the field $\mathbb{R}$. Here:

- data containers are real vector spaces $\mathbb{R}^{n_1\times\cdots\times n_k}$, equipped with the Euclidean topology and a natural smooth manifold structure;
- layer operations are smooth maps (or piecewise smooth, for ReLU activations);
- gradients, Jacobians, and the backpropagation chain rule are defined in the usual sense of differential calculus.

The gap. Hardware computes functions between finite sets (Level 1). Theory analyzes smooth maps between real vector spaces (Level 2). These are genuinely different mathematical objects. The claim that hardware arithmetic reliably approximates real arithmetic is a theorem of numerical analysis (Wilkinson's backward error analysis, floating-point error bounds) — it is *not* an assumption embedded in our mathematical model. We do not assume that $\mathbb{F}_{32}$ "approximates" $\mathbb{R}$; we simply note that both levels exist and state explicitly which level each result belongs to.

When we write $f:\mathbb{R}^{n\times d}\to\mathbb{R}^{n\times d}$ and say f is smooth, that is a Level 2 statement. When a GPU executes f using `float32`, that is a Level 1 computation in $\mathbb{F}_{32}^{n\times d}$.

2. Data Containers: Arrays, Matrices, Tensors

Definition 2.1 (Shape and the space $\mathbb{R}^{n_1\times\cdots\times n_k}$)**.** A *shape* is a k-tuple $(n_1,\dots,n_k)\in\mathbb{N}^k$ for some $k\geq 1$. The associated *array space* is

$$\mathbb{R}^{n_1\times\cdots\times n_k} \;:=\; \{A:\{0,\dots,n_1-1\}\times\cdots\times\{0,\dots,n_k-1\}\to\mathbb{R}\}.$$

This is a real vector space of dimension $n_1\cdots n_k$ under pointwise addition and scalar multiplication.

Remark 2.2 (Relation to tensor products)**.** There is a canonical isomorphism of real vector spaces

$$\mathbb{R}^{n_1\times\cdots\times n_k} \;\cong\; \mathbb{R}^{n_1}\otimes_{\mathbb{R}}\cdots\otimes_{\mathbb{R}}\mathbb{R}^{n_k},$$

via the identification of the standard basis element $e_{i_1,\dots,i_k}$ with $e_{i_1}\otimes\cdots\otimes e_{i_k}$. In this sense the ML usage of the word "tensor" (a multidimensional array) coincides, after a choice of basis, with the algebraic tensor product. The two notions diverge in coordinate-free settings (e.g. when discussing covariance/contravariance), but for our purposes the identification is exact.

Definition 2.3 (Matrix space $\mathrm{M}_{m\times n}(F)$)**.** Let $m,n\in\mathbb{N}$ and let F be any set.

$$\mathrm{M}_{m\times n}(F) \;:=\; \{A:\{1,\dots,m\}\times\{1,\dots,n\}\to F\},$$

the set of all functions from the index set $\{1,\dots,m\}\times\{1,\dots,n\}$ to F. An element $A\in\mathrm{M}_{m\times n}(F)$ is an $m\times n$ *matrix with coefficients in* F; we write $A_{ij}=A(i,j)$.

The notation:

- M stands for *matrix* (standard algebraic notation, following Bourbaki).
- **subscript** $m \times n$: m rows, n columns.
- (F): the *coefficient set* — the set to which each entry A_{ij} belongs, i.e. $A_{ij} \in F$. Depending on what F is (a field, a ring, a finite set with no algebraic structure), the matrix space $\mathrm{M}_{m\times n}(F)$ inherits exactly as much algebraic structure as F possesses (see the table below).

The algebraic structure $\mathrm{M}_{m\times n}(F)$ inherits from F is exactly as rich as F itself:

F **is ...**	$\mathrm{M}_{m\times n}(F)$ **is ...**
a field (e.g. $\mathbb{R}$, $\mathbb{Q}$)	a free F-module of rank mn; equivalently, an F-vector space of dimension mn.
a commutative ring (e.g. $\mathbb{Z}$, $\mathbb{Z}/2^k\mathbb{Z}$)	a free R-module of rank mn; pointwise addition and scalar multiplication satisfy all module axioms.
a non-commutative ring	a free left (or right) R-module; row-by-column multiplication of matrices with square F-entries is defined.
a semiring (e.g. $\mathbb{N}$, tropical $\mathbb{R}$)	a free F-semimodule (additive inverses not guaranteed).
a commutative non-associative magma with identity (e.g. $\mathbb{F}_{32}$ with IEEE 754 $\oplus$)	a set equipped with a binary operation $A \oplus B$ (coordinatewise); no module axioms hold (addition is not associative, so no abelian group structure).
an arbitrary set	just a set; no algebraic structure at all.

In these notes: At Level 2 (mathematical model, Remark 1.4) we always take $F = \mathbb{R}$, so $\mathrm{M}_{m\times n}(\mathbb{R})$ is a real vector space. We use the shorthand $\mathrm{M}_{m\times n}(\mathbb{R}) := \mathrm{M}_{m\times n}(\mathbb{R})$.

At Level 1 (computation), $F = \mathbb{F}_{32}$ (float32) makes $\mathrm{M}_{m\times n}(\mathbb{F}_{32})$ a finite set with no guaranteed algebraic structure; $F = \mathbb{Z}/2^k\mathbb{Z}$ (integer) makes $\mathrm{M}_{m\times n}(\mathbb{Z}/2^k\mathbb{Z})$ a free $\mathbb{Z}/2^k\mathbb{Z}$-module.

Remark 2.4 (What "vector" means here)**.** We reserve the word *vector* for an element of any real vector space, not specifically a 1-d array. A matrix $A \in \mathrm{M}_{m\times n}(\mathbb{R})$ is simultaneously:

- a vector in the vector space $\mathbb{R}^{m\times n} \cong \mathrm{M}_{m\times n}(\mathbb{R})$ (dimension mn);
- an $\mathbb{R}$-linear map $\mathbb{R}^n \to \mathbb{R}^m$ via $v \mapsto Av$ (acting on column vectors on the right);
- an element of the tensor product $(\mathbb{R}^n)^* \otimes_{\mathbb{R}} \mathbb{R}^m$, via the canonical isomorphism $L(\mathbb{R}^n, \mathbb{R}^m) \cong (\mathbb{R}^n)^* \otimes \mathbb{R}^m$ given by $\phi \otimes w \mapsto (v \mapsto \phi(v)\, w)$.

We use whichever view is most convenient, and will say explicitly which structure we are invoking.

Definition 2.5 (Standard data shapes in a Transformer)**.** Fix $B, n, d_{\text{model}}, d_k, d_v, h \in \mathbb{N}$ where:

- B = batch size,
- n = sequence length (number of tokens),
- d_{model} = model dimension (embedding dimension),
- d_k, d_v = key/value head dimensions,
- h = number of attention heads.

The standard data containers and their types are:

Name	Type	Role
Input sequence (one batch)	$\mathbb{R}^{B\times n\times d_{\text{model}}}$	token embeddings + positional enc.
Query / Key / Value (one head)	$\mathbb{R}^{B\times n\times d_k}$ or $\mathbb{R}^{B\times n\times d_v}$	projected inputs
Score matrix (one head)	$\mathbb{R}^{B\times n\times n}$	$QK^\top/\sqrt{d_k}$
Attention weight matrix	$\mathbb{R}^{B\times n\times n}$	after row-wise softmax
Output (one head)	$\mathbb{R}^{B\times n\times d_v}$	weighted sum of values
Weight matrix W_ℓ^Q	$\mathbb{R}^{d_{\text{model}}\times d_k}$	query projection, head ℓ

All of these are elements of real vector spaces. None of them is "a vector" in the colloquial sense of a 1-d list; all of them are vectors in the mathematical sense (elements of a vector space over $\mathbb{R}$).

3. What Is a Layer?

We now give a precise definition of what a "layer" is as a mathematical object, separating the *type of the data* from the *operation*.

Definition 3.1 (Parameterized map (a layer))**.** Let X and Y be finite-dimensional real vector spaces, and let Θ be a smooth manifold (the *parameter space*). A *layer* is a smooth map

$$f \;:\; \Theta \times X \;\longrightarrow\; Y.$$

For each fixed $\theta \in \Theta$, the map $f_\theta := f(\theta, \cdot) : X \to Y$ is the *forward pass* of the layer at parameters θ.

Remark 3.2 (Three distinct objects)**.** There are three distinct mathematical objects in play:

(i) The **input** $x \in X$: an element of a real vector space.
(ii) The **parameters** $\theta \in \Theta$: an element of a smooth manifold (in practice $\Theta = \mathbb{R}^p$ for some p, so it is also a real vector space).
(iii) The **layer** $f : \Theta \times X \to Y$: a smooth map. This is the "operation." It is *not* an element of X; it is a function.

In ML code, a "layer" is typically an object that stores θ and implements the map $x \mapsto f_\theta(x)$. Mathematically these are the same: the object is the pair (θ, f) where f is the smooth map.

Remark 3.3 (Parameter spaces in practice)**.** For a weight matrix $W \in \mathrm{M}_{m\times n}(\mathbb{R})$, the parameter space is $\Theta = \mathbb{R}^{m\times n}$ (a real vector space of dimension mn, hence also a smooth manifold diffeomorphic to $\mathbb{R}^{mn}$). A linear layer with weight W and bias $b \in \mathbb{R}^m$ has parameter space $\Theta = \mathbb{R}^{m\times n} \times \mathbb{R}^m$, also a vector space.

The *attention* operation (Section 6) takes W^Q, W^K, W^V, W^O as parameters and x (the sequence) as input. The parameter space is $\Theta_{\text{MHA}} = \mathbb{R}^{d_{\text{model}}\times d_k} \times \mathbb{R}^{d_{\text{model}}\times d_k} \times \mathbb{R}^{d_{\text{model}}\times d_v} \times \mathbb{R}^{hd_v\times d_{\text{model}}}$ (one tuple of weight matrices per head, plus the output projection).

Definition 3.4 (Composition of layers)**.** Let $f : \Theta_1 \times X \to Y$ and $g : \Theta_2 \times Y \to Z$ be layers. Their *composition* is the layer

$$g \circ f \;:\; (\Theta_1 \times \Theta_2) \times X \;\longrightarrow\; Z, \qquad (g \circ f)_{(\theta_1,\theta_2)}(x) \;:=\; g_{\theta_2}(f_{\theta_1}(x)).$$

The parameter space of the composed layer is the product $\Theta_1 \times \Theta_2$.

Definition 3.5 (Para construction)**.** Let $(\mathcal{C}, \otimes, I)$ be a symmetric monoidal category. The *Para construction* $\mathbf{Para}(\mathcal{C})$ is the category whose data are:

- **Objects**: the same objects as $\mathcal{C}$.
- **Morphisms** $X \to Y$: pairs (Θ, f) where $\Theta \in \mathcal{C}$ is an object (the *parameter object*) and $f : \Theta \otimes X \to Y$ is a morphism in $\mathcal{C}$.

- **Composition**: given $(\Theta_1,\ f : \Theta_1 \otimes X \to Y)$ and $(\Theta_2,\ g : \Theta_2 \otimes Y \to Z)$, their composite is

$$\big(\Theta_1 \otimes \Theta_2,\ h : (\Theta_1 \otimes \Theta_2) \otimes X \to Z\big), \qquad h_{(\theta_1,\theta_2)}(x) \ := \ g_{\theta_2}(f_{\theta_1}(x)).$$

- **Identity** on X: the pair $(I,\ \lambda_X)$ where $\lambda_X : I \otimes X \xrightarrow{\sim} X$ is the left-unit isomorphism of $\mathcal{C}$.

Remark 3.6 (Categorical structure: Para(Smooth))**.** Definition 3.4 is exactly the composition in the *Para* construction (Definition 3.5) applied to the category **Smooth** of smooth manifolds. The resulting category **Para**(**Smooth**) has:

- **Objects**: smooth manifolds (e.g. $\mathbb{R}^{n\times d}$ for various shapes).
- **Morphisms** $X \to Y$: pairs $(\Theta, f : \Theta \times X \to Y)$ for some smooth manifold Θ.
- **Composition**: as in Definition 3.4.

A *neural network* is a morphism in **Para**(**Smooth**): a composed sequence of layers. Training (gradient descent on the loss) is gradient flow on $\Theta_1 \times \cdots \times \Theta_L$ with respect to the loss function $\mathcal{L} : \Theta_1 \times \cdots \times \Theta_L \to \mathbb{R}$.

Reference: Cruttwell, Gavranović, Ghani, Wilson, Zanasi, "Categorical Foundations of Gradient-Based Learning," ESOP 2022.

Example 3.7 (Specific layers in the Transformer)**.** With the above definition, each component of the Transformer is a layer in the sense of Definition 3.1:

(1) **Linear projection**: $f_W(x) = xW$ where $\Theta = \mathrm{M}_{d_{\text{in}}\times d_{\text{out}}}(\mathbb{R})$, $X = \mathbb{R}^{n\times d_{\text{in}}}$, $Y = \mathbb{R}^{n\times d_{\text{out}}}$. This is a bilinear map in (W, x), hence smooth.
(2) **Scaled dot-product attention**: parameter-free for a single call to $\mathrm{Att}(Q, K, V)$, so $\Theta = \{*\}$ (a point) and $\mathrm{Att} : \mathrm{M}_{n\times d_k}(\mathbb{R})^2 \times \mathrm{M}_{n\times d_v}(\mathbb{R}) \to \mathrm{M}_{n\times d_v}(\mathbb{R})$ is a smooth map on the input. The weight matrices W^Q, W^K, W^V belong to the parameter space of the enclosing multi-head attention layer.
(3) **Layer normalisation**: $\Theta = \mathbb{R}^d \times \mathbb{R}^d$ (the learnable scale γ and shift β), $X = Y = \mathbb{R}^d$. The map is smooth away from the set $\{x : x = c\mathbf{1} \text{ for some } c \in \mathbb{R}\}$ (zero variance).
(4) **Feed-forward sublayer**: $\Theta = \mathrm{M}_{d\times d_{\text{ff}}}(\mathbb{R}) \times \mathbb{R}^{d_{\text{ff}}} \times \mathrm{M}_{d_{\text{ff}}\times d}(\mathbb{R}) \times \mathbb{R}^d$, the map $x \mapsto \max(0, xW_1 + b_1)W_2 + b_2$ is *piecewise smooth* (smooth on each region of the partition defined by the activation thresholds) but not globally smooth.
(5) **Softmax**: parameter-free, $\Theta = \{*\}$, $\mathrm{softmax} : \mathbb{R}^n \to \mathrm{int}(\Delta^{n-1})$ is smooth (Section 5).

Remark 3.8 (Set membership is how mathematics says what an object is)**.** In classical mathematics (ZFC set theory), the primitive notion is *set membership*: to say what a mathematical object is, one states the set it belongs to, written $x \in S$. There is no separate notion of "type" — that is a construct from type theory (Martin-Löf, Curry-Howard) and programming languages, a different foundational framework from set theory. Using "type" in a classical-mathematical context imports an unnecessary and confusing foreign concept when "$x \in S$" suffices completely.

Concretely, for the objects in this document:

- The input sequence: $x \in X$, where $X = \mathbb{R}^{B\times n\times d}$ is a real vector space.
- The parameters: $\theta \in \Theta$, where $\Theta = \mathbb{R}^p$ is a smooth manifold.
- A layer: $f \in C^\infty(\Theta \times X, Y)$, i.e. f is an element of the set of smooth maps from $\Theta \times X$ to Y.

The critical observation is that $x \in X$ and $f \in C^\infty(\Theta \times X, Y)$ are elements of *different sets*: x is a point in a vector space; f is a smooth map between vector spaces. In ML frameworks both are called "tensors," which conflates this distinction. The mathematical statement $x \in X$ and $f \in C^\infty(\Theta \times X, Y)$ carries all the necessary information without any auxiliary notion of "type."

Part 2. Paper I: The Attention Mechanism

Source: Vaswani, Shazeer, Parmar, Uszkoreit, Jones, Gomez, Kaiser, Polosukhin. "Attention Is All You Need." NeurIPS 2017. arXiv:1706.03762.

The paper introduces the *Transformer*, a sequence-to-sequence architecture built entirely from attention layers and pointwise feed-forward layers, with no recurrence or convolution. Our treatment extracts the mathematical content — the definitions, the geometry of softmax, the permutation equivariance of attention, and the role of positional encoding — and presents it independently of the machine translation motivation.

4. Setup: Sequences as Matrix Rows

Throughout, $d_{\text{model}}, d_k, d_v, n \in \mathbb{N}$ are positive integers.

Definition 4.1 (Sequence)**.** A *sequence* of length n in $\mathbb{R}^d$ is an element of $\mathbb{R}^{n\times d}$, viewed as a matrix whose i-th row is the i-th element of the sequence.

Remark 4.2 ("Projected" and "learned")**.** No inner product on $\mathbb{R}^{d_{\text{model}}}$ is fixed globally; attention dot products operate in $\mathbb{R}^{d_k}$. The paper (§3.2) says the sequence is "projected by learned weight matrices" and calls W^Q, W^K, W^V "projection matrices."

"**Projected**" (§3.2) ⟨*precisely:* right-multiplied on the right: $Q = XW^Q$ where $X \in \mathrm{M}_{n\times d_{\text{model}}}(\mathbb{R})$, $W^Q \in \mathrm{M}_{d_{\text{model}}\times d_k}(\mathbb{R})$, $Q \in \mathrm{M}_{n\times d_k}(\mathbb{R})$; i.e. the $\mathbb{R}$-linear map $R_{W^Q} : X \mapsto XW^Q$ (matrix–matrix multiplication)⟩. Note: in algebra, a *projection* means $P^2 = P$ (idempotent); these weight matrices need not satisfy this, and squaring is not even defined when $d_k \neq d_{\text{model}}$. The paper uses "projection" to mean "linear map."

"**Learned**" (§3) ⟨*precisely:* $W^Q \in \Theta$ is a parameter optimised during training, not a fixed geometric object⟩.

5. The Softmax Map

Definition 5.1 (Standard simplex)**.** The *standard* $(n-1)$*-simplex* is

$$\Delta^{n-1} \;:=\; \bigl\{p \in \mathbb{R}^n \;\big|\; p_i \geq 0,\ \textstyle\sum_{i=1}^n p_i = 1\bigr\}.$$

Its *interior* is $\operatorname{int}(\Delta^{n-1}) = \{p \in \Delta^{n-1} \mid p_i > 0 \text{ for all } i\}$.

Definition 5.2 (Log-sum-exp and softmax)**.** The *log-sum-exp* function $\operatorname{lse} : \mathbb{R}^n \to \mathbb{R}$ is

$$\operatorname{lse}(x) \;:=\; \log \sum_{i=1}^n e^{x_i}.$$

The *softmax* map $\operatorname{softmax} : \mathbb{R}^n \to \operatorname{int}(\Delta^{n-1})$ is

$$\operatorname{softmax}(x)_i \;:=\; \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}} \;=\; e^{x_i - \operatorname{lse}(x)}, \qquad i = 1, \ldots, n.$$

Proposition 5.3 (Softmax as gradient of log-partition function)**.** $\operatorname{softmax} = \nabla \operatorname{lse}$, *i.e.* $\operatorname{softmax}(x)_i = \frac{\partial}{\partial x_i} \operatorname{lse}(x)$.

Proof. Direct computation: $\frac{\partial}{\partial x_i} \log \sum_j e^{x_j} = \frac{e^{x_i}}{\sum_j e^{x_j}} = \operatorname{softmax}(x)_i$. □

Remark 5.4. The function lse is the *log-partition function* (free energy) of the Gibbs measure on $\{1, \ldots, n\}$ with energy $-x$. Its Legendre transform is the negative entropy $\sum_i p_i \log p_i$ on Δ^{n-1}. Proposition 5.3 is the statement that softmax is the Legendre dual variable map.

Proposition 5.5 (Diffeomorphism to simplex interior)**.** $\mathrm{softmax} : \mathbb{R}^n \to \mathrm{int}(\Delta^{n-1})$ *is a smooth surjection. Its fibers are the affine lines* $\{x + t\mathbf{1} \mid t \in \mathbb{R}\}$*, i.e.* $\mathrm{softmax}(x) = \mathrm{softmax}(y)$ *if and only if* $x - y \in \mathbb{R}\mathbf{1}$*. Consequently* softmax *descends to a diffeomorphism*

$$\widetilde{\mathrm{softmax}} : \mathbb{R}^n/\mathbb{R}\mathbf{1} \xrightarrow{\sim} \mathrm{int}(\Delta^{n-1}).$$

Proof. Smoothness is clear. The fiber condition follows from $\mathrm{softmax}(x) = \mathrm{softmax}(y) \Leftrightarrow e^{x_i}/e^{x_j} = e^{y_i}/e^{y_j}$ for all i, j, i.e. $x_i - x_j = y_i - y_j$ for all i, j, i.e. $x - y \in \mathbb{R}\mathbf{1}$. The quotient $\mathbb{R}^n/\mathbb{R}\mathbf{1} \cong \mathbb{R}^{n-1}$ is $(n-1)$-dimensional, as is $\mathrm{int}(\Delta^{n-1})$. Surjectivity and the inverse map are explicit: given $p \in \mathrm{int}(\Delta^{n-1})$, any x with $x_i = \log p_i$ satisfies $\mathrm{softmax}(x) = p$. □

Proposition 5.6 (Translation invariance)**.** *For any* $m \in \mathbb{R}$ *and* $x \in \mathbb{R}^n$*:*

$$\mathrm{softmax}(x)_i \;=\; \mathrm{softmax}(x - m\mathbf{1})_i \;=\; \frac{e^{x_i - m}}{\sum_j e^{x_j - m}}.$$

This proposition is numerically crucial: subtracting the row-maximum $m = \max_j x_j$ before exponentiating prevents overflow, without changing the output. This is exploited in the online softmax algorithm (Paper II).

6. Scaled Dot-Product Attention

Definition 6.1 (Attention inputs)**.** Fix sequence length n, key/query dimension d_k, value dimension d_v. The *query*, *key*, and *value* matrices are

$$Q \in \mathrm{M}_{n \times d_k}(\mathbb{R}), \quad K \in \mathrm{M}_{n \times d_k}(\mathbb{R}), \quad V \in \mathrm{M}_{n \times d_v}(\mathbb{R}).$$

Row i of Q is the query vector for position i; row j of K (resp. V) is the key (resp. value) vector for position j.

Definition 6.2 (Scaled dot-product attention)**.** The *score matrix* is

$$S \;:=\; \frac{QK^\top}{\sqrt{d_k}} \;\in\; \mathrm{M}_{n \times n}(\mathbb{R}).$$

Row i of S is $S_i = \frac{1}{\sqrt{d_k}} K q_i^\top \in \mathbb{R}^n$, where $q_i \in \mathbb{R}^{d_k}$ is row i of Q.

The *attention weight matrix* is $P \in \mathrm{M}_{n \times n}(\mathbb{R})$ with

$$P_i \;:=\; \mathrm{softmax}(S_i) \;\in\; \mathrm{int}(\Delta^{n-1}),$$

where P_i denotes row i of P. Equivalently, $P_{ij} = e^{S_{ij}} / \sum_k e^{S_{ik}}$.

The *attention output* is

$$O \;:=\; PV \;\in\; \mathrm{M}_{n \times d_v}(\mathbb{R}).$$

The *scaled dot-product attention* map is

$$\mathrm{Att} : \mathrm{M}_{n \times d_k}(\mathbb{R}) \times \mathrm{M}_{n \times d_k}(\mathbb{R}) \times \mathrm{M}_{n \times d_v}(\mathbb{R}) \;\longrightarrow\; \mathrm{M}_{n \times d_v}(\mathbb{R})$$

$$\mathrm{Att}(Q, K, V) \;:=\; \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V,$$

where softmax is applied row-wise.

Remark 6.3 (Interpretation)**.** Row i of O is

$$O_i \;=\; \sum_{j=1}^{n} P_{ij}\, v_j \;\in\; \mathrm{conv}\{v_1, \ldots, v_n\} \;\subset\; \mathbb{R}^{d_v},$$

a convex combination of the value vectors, with weights determined by the similarity of query q_i to each key k_j. Geometrically, attention is a *soft nearest-neighbour lookup*: position i retrieves a weighted average of values, weighted by the probability that key k_j is the best match for query q_i.

Remark 6.4 (Jacobian of softmax)**.** For $\sigma = \mathrm{softmax}(x) \in \mathbb{R}^n$, the Jacobian is

$$J\,\mathrm{softmax}(x) \;=\; \mathrm{diag}(\sigma) - \sigma\sigma^\top \;\in\; \mathrm{M}_{n\times n}(\mathbb{R}).$$

Derivation. Write $Z := \sum_{k=1}^n e^{x_k}$ and $\sigma_j = e^{x_j}/Z$. Differentiate σ_j with respect to x_l by the quotient rule:

$$\frac{\partial \sigma_j}{\partial x_l} \;=\; \frac{\delta_{jl}\, e^{x_j}\cdot Z \;-\; e^{x_j}\cdot e^{x_l}}{Z^2} \;=\; \frac{e^{x_j}}{Z}\,\delta_{jl} \;-\; \frac{e^{x_j}}{Z}\cdot\frac{e^{x_l}}{Z} \;=\; \sigma_j\,\delta_{jl} - \sigma_j\,\sigma_l \;=\; \sigma_j(\delta_{jl}-\sigma_l).$$

Entry (j,l) of the Jacobian is therefore

$$\big(J\,\mathrm{softmax}(x)\big)_{jl} \;=\; \begin{cases} \sigma_j(1-\sigma_j) & j = l, \\ -\sigma_j\,\sigma_l & j \neq l, \end{cases}$$

which in matrix form is $(\mathrm{diag}(\sigma))_{jl} - (\sigma\sigma^\top)_{jl}$, giving $J\,\mathrm{softmax}(x) = \mathrm{diag}(\sigma) - \sigma\sigma^\top$.

This matrix is the Fisher information metric on $\mathrm{int}(\Delta^{n-1})$ and equals the covariance matrix $\mathrm{Cov}_\sigma[\mathrm{Id}]$ of the distribution σ (restricted to $T_\sigma\Delta^{n-1}$).

Proposition 6.5 (Smoothness and Jacobian w.r.t. Q)**.** Att *is a smooth map. The Jacobian with respect to Q is block-diagonal: output row $O_{i,\cdot}$ depends only on query row $Q_{i,\cdot}$ (not on any $Q_{l,\cdot}$ with $l \neq i$), with $(d_v \times d_k)$-block*

$$\frac{\partial O_{i,\cdot}}{\partial Q_{i,\cdot}} \;=\; \frac{1}{\sqrt{d_k}}\,V^\top\big(\mathrm{diag}(p_i) - p_i p_i^\top\big)K \;\in\; \mathrm{M}_{d_v\times d_k}(\mathbb{R}),$$

where $p_i \in \mathbb{R}^n$ is row i of P viewed as a column vector.

Proof. Smoothness. Att is the composition of: $(Q,K) \mapsto QK^\top/\sqrt{d_k}$ (bilinear, smooth); $S \mapsto \mathrm{softmax}_{\mathrm{rows}}(S)$ (smooth, each row being $\nabla\,\mathrm{lse}$, which is C^∞); $P \mapsto PV$ (linear, smooth).

Block-diagonal structure. Set $s_i := Q_{i,\cdot}K^\top/\sqrt{d_k} \in \mathbb{R}^n$ (row i of S) and $p_i := \mathrm{softmax}(s_i)$ (row i of P). Because s_i depends on $Q_{i,\cdot}$ only, and $O_{i,\cdot} = V^\top p_i$ depends on P only through p_i, we have $\partial O_{i,\cdot}/\partial Q_{l,\cdot} = 0$ for $l \neq i$.

Diagonal block, entry by entry. Fix row i; write $q := Q_{i,\cdot} \in \mathbb{R}^{d_k}$. Since $s_{ij} = q\cdot k_j/\sqrt{d_k}$ where k_j is row j of K:

$$\frac{\partial s_{ij}}{\partial q_b} \;=\; \frac{K_{jb}}{\sqrt{d_k}}.$$

Apply the chain rule through softmax (Remark 6.4, $(J\,\mathrm{softmax})_{jc} = p_{ij}(\delta_{jc}-p_{ic})$):

$$\frac{\partial p_{ij}}{\partial q_b} \;=\; \sum_{c=1}^n \frac{\partial p_{ij}}{\partial s_{ic}}\cdot\frac{\partial s_{ic}}{\partial q_b} \;=\; \sum_c p_{ij}(\delta_{jc}-p_{ic})\,\frac{K_{cb}}{\sqrt{d_k}} \;=\; \frac{p_{ij}}{\sqrt{d_k}}\big(K_{jb} - (PK)_{ib}\big),$$

where $(PK)_{ib} = \sum_c p_{ic}K_{cb}$ is entry (i,b) of PK. Now multiply by V_{ja} and sum over j:

$$\begin{aligned} \frac{\partial O_{ia}}{\partial q_b} \;&=\; \sum_{j=1}^n \frac{\partial p_{ij}}{\partial q_b}\,V_{ja} \\ &=\; \frac{1}{\sqrt{d_k}}\sum_j p_{ij}\,V_{ja}\big(K_{jb} - (PK)_{ib}\big) \\ &=\; \frac{1}{\sqrt{d_k}}\left[\underbrace{\sum_j p_{ij}\,V_{ja}\,K_{jb}}_{(V^\top\mathrm{diag}(p_i)K)_{ab}} \;-\; \underbrace{\Big(\sum_j p_{ij}\,V_{ja}\Big)}_{O_{ia}=(V^\top p_i)_a}\underbrace{(PK)_{ib}}_{(p_i^\top K)_b}\right]. \end{aligned}$$

The first bracket is $(V^\top \operatorname{diag}(p_i)K)_{ab}$; the second is $(V^\top p_i\, p_i^\top K)_{ab}$. Therefore

$$\frac{\partial O_{ia}}{\partial q_b} \;=\; \frac{1}{\sqrt{d_k}}\big[V^\top(\operatorname{diag}(p_i) - p_i p_i^\top)K\big]_{ab}. \qquad \square$$

7. The Scaling Factor $1/\sqrt{d_k}$

Proposition 7.1 (Variance of dot products)**.** *Let $q, k \in \mathbb{R}^{d_k}$ be random vectors whose coordinates are independent with mean 0 and variance 1. Then*

$$\mathbb{E}[\langle q,\, k\rangle] \;=\; 0, \qquad \operatorname{Var}[\langle q,\, k\rangle] \;=\; d_k.$$

Proof. $\langle q,\, k\rangle = \sum_{i=1}^{d_k} q_i k_i$. By independence of q_i and k_i, $\mathbb{E}[q_i k_i] = \mathbb{E}[q_i]\mathbb{E}[k_i] = 0$. The variance is $\operatorname{Var}[q_i k_i] = \mathbb{E}[q_i^2 k_i^2] - 0 = 1 \cdot 1 = 1$, so $\operatorname{Var}[\langle q,\, k\rangle] = \sum_i 1 = d_k$. $\square$

Corollary 7.2. *The scaled score $\langle q,\, k\rangle/\sqrt{d_k}$ has mean 0 and variance 1. Dividing by $\sqrt{d_k}$ prevents the dot products from growing large in magnitude as d_k increases, which would push the softmax output toward the vertices of Δ^{n-1} (near-zero gradient regime).*

8. Permutation Equivariance of Attention

Remark 8.1 (Why we study permutation equivariance)**.** Scaled dot-product attention computes scores $S_{ij} = q_i \cdot k_j/\sqrt{d_k}$ symmetrically in the index j: there is no formula that says "position 1 is special" or "position 3 comes after position 2." Formalising this symmetry tells us exactly what information attention *cannot* represent, and therefore what must be added externally (positional encodings, §9) to break it. Permutation equivariance is also a useful sanity check on any new attention variant: if a proposed change breaks equivariance, that change has introduced an implicit positional bias.

Definition 8.2 (Symmetric group S_n)**.** Let $n \in \mathbb{N}$. The *symmetric group* S_n is the group of all bijections from the set $\{1, \ldots, n\}$ to itself, with group operation given by composition of functions:

$$(\pi \circ \tau)(i) \;:=\; \pi(\tau(i)), \qquad \pi, \tau \in S_n, \quad i \in \{1, \ldots, n\}.$$

Identity: $\mathrm{id}(i) = i$. Inverse: the function inverse π^{-1}. Order: $|S_n| = n!$ (there are $n!$ bijections of an n-element set).

An element $\pi \in S_n$ is a bijection, not a letter. The set $\{1, \ldots, n\}$ is the *domain and codomain* of π — the n objects being permuted. An element $\pi \in S_n$ is a function $\pi : \{1, \ldots, n\} \to \{1, \ldots, n\}$; it is *not* one of the symbols $1, \ldots, n$ themselves. For $n = 3$, the cyclic element $\pi = (1 \mapsto 2, 2 \mapsto 3, 3 \mapsto 1)$ is one of the $6 = 3!$ bijections in S_3; the number $1 \in \{1, 2, 3\}$ is an element of the set on which π acts.

Generalisation to any n-element set. The choice of $\{1, \ldots, n\}$ is a labeling convention. For any set X with $|X| = n$ — e.g. $X = \{a, b, c\}$, or $X =$ the tokens of a sentence — the group $\operatorname{Sym}(X) := \{\text{bijections } X \to X\}$ is isomorphic to S_n: any bijection $\phi : X \xrightarrow{\sim} \{1, \ldots, n\}$ gives an isomorphism $\operatorname{Sym}(X) \to S_n$, $f \mapsto \phi \circ f \circ \phi^{-1}$. So S_n describes the symmetries of *any* n-element set.

Each $\pi \in S_n$ acts on $\mathrm{M}_{n\times d}(\mathbb{R})$ by left-permuting rows: $(\pi \cdot A)_i := A_{\pi^{-1}(i)}$. (The inverse appears so that the map $(\pi, A) \mapsto \pi \cdot A$ is a left group action: $\pi \cdot (\tau \cdot A) = (\pi \circ \tau) \cdot A$.)

Proposition 8.3 (Equivariance)**.** *For any $\pi \in S_n$ and any Q, K, V:*

$$\mathrm{Att}(\pi \cdot Q,\; \pi \cdot K,\; \pi \cdot V) \;=\; \pi \cdot \mathrm{Att}(Q, K, V).$$

In other words, Att *is S_n-equivariant.*

Proof. Let $P_\pi \in \mathrm{M}_{n\times n}(\mathbb{R})$ be the permutation matrix for π. Then $\pi \cdot Q = P_\pi Q$, and similarly for K, V. The score matrix transforms as

$$S' \;=\; \frac{(P_\pi Q)(P_\pi K)^\top}{\sqrt{d_k}} \;=\; P_\pi \frac{QK^\top}{\sqrt{d_k}} P_\pi^\top \;=\; P_\pi S P_\pi^\top.$$

Row-wise softmax commutes with row-permutation: $\mathrm{softmax}(P_\pi S P_\pi^\top)_i = \mathrm{softmax}((P_\pi S P_\pi^\top)_i) = \mathrm{softmax}(S_{\pi^{-1}(i)} P_\pi^\top)$. Since $P_\pi^\top$ permutes columns, and softmax is translation-invariant up to a permuted normalizer, one checks that $P = \mathrm{softmax}(S')$ satisfies $P = P_\pi\,\mathrm{softmax}(S) P_\pi^\top$. Therefore

$$O' \;=\; P\,(P_\pi V) \;=\; P_\pi\,\mathrm{softmax}(S) P_\pi^\top P_\pi V \;=\; P_\pi\,\mathrm{softmax}(S) V \;=\; P_\pi O. \qquad \square$$

Corollary 8.4. *Pure attention is* set-valued*: it treats the input as an unordered multiset of vectors. Two sequences that are permutations of each other produce outputs that are the same permutation of each other. In particular, a pure attention layer carries no information about the* order *of the sequence.*

This is the mathematical reason positional encodings must be added.

9. Positional Encoding

To break the S_n-equivariance, the paper adds a fixed encoding $\mathrm{PE} \in \mathrm{M}_{n\times d_{\text{model}}}(\mathbb{R})$ to the embedded input sequence before any attention layers.

Definition 9.1 (Sinusoidal positional encoding)**.** For $\mathrm{pos} = 1, \ldots, n$ and $i = 0, \ldots, d_{\text{model}}/2 - 1$:

$$\begin{aligned} \mathrm{PE}_{\mathrm{pos},\,2i} \;&:=\; \sin\!\Big(\frac{\mathrm{pos}}{10000^{2i/d_{\text{model}}}}\Big), \\ \mathrm{PE}_{\mathrm{pos},\,2i+1} \;&:=\; \cos\!\Big(\frac{\mathrm{pos}}{10000^{2i/d_{\text{model}}}}\Big). \end{aligned}$$

Proposition 9.2 (Relative encoding via rotation)**.** *Fix a frequency index i. The pair $(\mathrm{PE}_{\mathrm{pos},2i},\, \mathrm{PE}_{\mathrm{pos},2i+1}) = (\sin(\omega_i\,\mathrm{pos}),\, \cos(\omega_i\,\mathrm{pos}))$ where $\omega_i = 10000^{-2i/d_{\text{model}}}$.*

For any fixed offset $k \in \mathbb{N}$, the vector $\mathrm{PE}_{\mathrm{pos}+k}$ in the $2i$-th and $(2i+1)$-th coordinates is obtained from $\mathrm{PE}_{\mathrm{pos}}$ by the rotation matrix $R(\omega_i k) \in \mathrm{SO}(2)$:

$$\begin{pmatrix} \mathrm{PE}_{\mathrm{pos}+k,\,2i} \\ \mathrm{PE}_{\mathrm{pos}+k,\,2i+1} \end{pmatrix} \;=\; \begin{pmatrix} \cos(\omega_i k) & -\sin(\omega_i k) \\ \sin(\omega_i k) & \cos(\omega_i k) \end{pmatrix} \begin{pmatrix} \mathrm{PE}_{\mathrm{pos},\,2i} \\ \mathrm{PE}_{\mathrm{pos},\,2i+1} \end{pmatrix}.$$

Proof. This is the angle-addition formula: $\sin(\omega_i(\mathrm{pos}+k)) = \sin(\omega_i\,\mathrm{pos})\cos(\omega_i k) + \cos(\omega_i\,\mathrm{pos})\sin(\omega_i k)$, and similarly for cosine. $\square$

Remark 9.3. Proposition 9.2 means that *relative position k can be computed linearly from absolute positions*, enabling the model to generalize to relative offsets. The $d_{\text{model}}/2$ different frequencies ω_i form a geometric progression $1, 10000^{-2/d}, 10000^{-4/d}, \ldots$, spanning wavelengths from 2π (high frequency, adjacent positions) to $10000 \cdot 2\pi$ (low frequency, distant positions). This is a discrete Fourier-like decomposition of position.

10. Multi-Head Attention

Rather than a single attention map, Vaswani et al. use h independent *heads*, each operating after right-multiplying the input by a different learned weight matrix (in the informal language of the paper: "in a lower-dimensional projected subspace"; see Remark 4.2).

Definition 10.1 (Multi-head attention)**.** Fix $h \in \mathbb{N}$ (number of heads), $d_k, d_v \in \mathbb{N}$ (head dimensions). For each head $\ell = 1, \ldots, h$, fix weight matrices

$$W_\ell^Q \in \mathrm{M}_{d_{\text{model}}\times d_k}(\mathbb{R}), \quad W_\ell^K \in \mathrm{M}_{d_{\text{model}}\times d_k}(\mathbb{R}), \quad W_\ell^V \in \mathrm{M}_{d_{\text{model}}\times d_v}(\mathbb{R}),$$

and an output weight matrix $W^O \in \mathrm{M}_{hd_v \times d_{\text{model}}}(\mathbb{R})$. (The paper calls these "projection matrices"; as noted in Remark 4.2, each defines a right-multiplication linear map, not an idempotent projection.)

For input sequences $Q, K, V \in \mathrm{M}_{n \times d_{\text{model}}}(\mathbb{R})$, define

$$\begin{aligned} \text{head}_\ell &:= \text{Att}(QW_\ell^Q,\ KW_\ell^K,\ VW_\ell^V) \ \in\ \mathrm{M}_{n \times d_v}(\mathbb{R}), \\ \text{MHA}(Q,K,V) &:= \big[\text{head}_1 \parallel \cdots \parallel \text{head}_h\big] W^O \ \in\ \mathrm{M}_{n \times d_{\text{model}}}(\mathbb{R}), \end{aligned}$$

where $[\cdot \parallel \cdots \parallel \cdot]$ denotes column-wise concatenation (so the argument of W^O lies in $\mathrm{M}_{n \times hd_v}(\mathbb{R})$).

Remark 10.2 (Column-wise concatenation: explicit element definition)**.** "Column-wise" means the columns of each head are placed *side by side* (the result is *wider*, not taller). Precisely: the concatenated matrix

$$C \ := \ [\text{head}_1 \parallel \cdots \parallel \text{head}_h] \ \in\ \mathrm{M}_{n \times hd_v}(\mathbb{R})$$

has entries

$$C_{i,\,(\ell-1)d_v + j} \ := \ (\text{head}_\ell)_{ij}, \qquad i = 1, \ldots, n, \quad \ell = 1, \ldots, h, \quad j = 1, \ldots, d_v.$$

Row i of C is the *horizontal* join of row i from each head in order.

Explicit example. Take $n = 2$ (two sequence positions), $h = 2$ heads, $d_v = 2$ (value dimension per head). Each head is a 2×2 matrix:

$$\text{head}_1 = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}, \qquad \text{head}_2 = \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix}.$$

Column-wise concatenation places them side by side:

$$[\text{head}_1 \parallel \text{head}_2] \ = \ \left(\begin{array}{cc|cc} a_{11} & a_{12} & b_{11} & b_{12} \\ a_{21} & a_{22} & b_{21} & b_{22} \end{array}\right) \ \in\ \mathrm{M}_{2 \times 4}(\mathbb{R}).$$

The result has the same number of *rows* ($n = 2$) and twice as many *columns* ($hd_v = 4$).

What it is not. "Row-wise" or vertical stacking would produce a taller matrix:

$$\begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix} \ \in\ \mathrm{M}_{4 \times 2}(\mathbb{R}).$$

That is *not* what $[\cdot \parallel \cdot]$ means here: we keep n rows and grow the column count from d_v to hd_v.

Remark 10.3 (Dimension bookkeeping)**.** The paper uses $d_{\text{model}} = 512$, $h = 8$, $d_k = d_v = 64$. Then $h \cdot d_v = 8 \cdot 64 = 512 = d_{\text{model}}$, so the concatenated output has the same dimension as the input; $W^O \in \mathrm{M}_{512 \times 512}(\mathbb{R})$.

The total FLOPs of multi-head attention with these dimensions equals that of a single head with $d_k = d_v = d_{\text{model}}$, because $h \cdot (n \cdot d_k) = n \cdot d_{\text{model}}$ for the Q projection.

Remark 10.4 (Subspace decomposition)**.** Multi-head attention allows the model to attend to information from h different representation subspaces simultaneously. Concretely, the ℓ-th head applies attention in the image of W_ℓ^Q (as a subspace of $\mathbb{R}^{d_k}$), and the final W^O aggregates across heads. With a single head, averaging over all j produces a single convex combination, which cannot simultaneously encode several independent relations between positions.

11. Pointwise Feed-Forward Sublayers and Layer Normalisation

Definition 11.1 (Rectified Linear Unit (ReLU))**.** The *rectified linear unit* is the map $\mathrm{ReLU} : \mathbb{R} \to \mathbb{R}$ defined by

$$\mathrm{ReLU}(t) \;:=\; \max(0,\, t) \;=\; \begin{cases} t & t > 0, \\ 0 & t \le 0. \end{cases}$$

Applied componentwise to a vector $z \in \mathbb{R}^m$:

$$\mathrm{ReLU}(z)_j \;:=\; \max(0,\, z_j), \qquad j = 1, \dots, m.$$

ReLU is continuous and piecewise linear; it is differentiable on $\mathbb{R} \setminus \{0\}$ with derivative $\mathrm{ReLU}'(t) = \mathbf{1}_{t>0}$ (the indicator function of $t > 0$). It is *not* differentiable at $t = 0$.

Definition 11.2 (Position-wise fully connected feed-forward network)**.** Fix parameters $W_1 \in \mathrm{M}_{d_{\text{model}} \times d_{\text{ff}}}(\mathbb{R})$, $b_1 \in \mathbb{R}^{d_{\text{ff}}}$, $W_2 \in \mathrm{M}_{d_{\text{ff}} \times d_{\text{model}}}(\mathbb{R})$, $b_2 \in \mathbb{R}^{d_{\text{model}}}$.

The *per-position* map $f : \mathbb{R}^{d_{\text{model}}} \to \mathbb{R}^{d_{\text{model}}}$ is

$$f(x) \;:=\; \mathrm{ReLU}(xW_1 + b_1)\, W_2 + b_2,$$

where ReLU (Definition 11.1) is applied componentwise to $xW_1 + b_1 \in \mathbb{R}^{d_{\text{ff}}}$.

Applied to a full sequence $X \in \mathrm{M}_{n \times d_{\text{model}}}(\mathbb{R})$:

$$\mathrm{FFN}(X)_i \;:=\; f(X_i), \qquad i = 1, \dots, n,$$

i.e. the same f is applied to each row $X_i \in \mathbb{R}^{d_{\text{model}}}$ independently.

Remark 11.3 ("Position-wise" and "fully connected")**.** "**Position-wise**" (§3.3) ⟨*precisely:* the same map $f : \mathbb{R}^{d_{\text{model}}} \to \mathbb{R}^{d_{\text{model}}}$ applied to each row X_i independently: $\mathrm{FFN}(X) = [f(X_1); \dots; f(X_n)]$; row i uses only X_i, row $j \neq i$ plays no role⟩. Contrast with MHA, where each output row depends on all input rows.

"**Fully connected**" (§3.3) ⟨*precisely:* $W_1 \in \mathrm{M}_{d_{\text{model}} \times d_{\text{ff}}}(\mathbb{R})$ is a general dense matrix: every input dimension contributes to every hidden dimension, no sparsity pattern imposed⟩. As opposed to a convolution, where each output depends only on a local window of k input positions.

FFN is piecewise-affine: on each stratum of the partition of $\mathbb{R}^{d_{\text{model}}}$ defined by the sign pattern of $xW_1 + b_1$ (i.e. each region where ReLU is linear), f is an affine map; it is not smooth at the activation boundary $\{x : (xW_1 + b_1)_j = 0 \text{ for some } j\}$ (since ReLU is not differentiable at 0). The paper uses $d_{\text{ff}} = 2048 = 4\, d_{\text{model}}$.

Definition 11.4 (Layer normalisation)**.** For $x \in \mathbb{R}^d$, let $\mu(x) = \frac{1}{d}\sum_i x_i$ and $\sigma(x) = \sqrt{\frac{1}{d}\sum_i (x_i - \mu(x))^2 + \epsilon}$ (with $\epsilon > 0$ for numerical stability). With learned parameters $\gamma, \beta \in \mathbb{R}^d$:

$$\mathrm{LayerNorm}(x)_i \;:=\; \frac{x_i - \mu(x)}{\sigma(x)}\, \gamma_i + \beta_i.$$

Remark 11.5 (Geometry of layer norm)**.** The map $x \mapsto (x - \mu(x)\mathbf{1})/\sigma(x)$ first subtracts the mean ⟨*precisely:* orthogonal projection onto $\{z : \sum_i z_i = 0\}$, which IS idempotent ($P^2 = P$) — this is projection in the algebraic sense⟩, then normalises to unit length ⟨*precisely:* divides by $\|x - \mu(x)\mathbf{1}\|$, landing on the sphere S^{d-1}⟩, then rescales and shifts by the learned (γ, β). The full map is smooth on $\mathbb{R}^d \setminus \{\mu(x)\mathbf{1}\}$.

12. The Encoder Stack

Each encoder layer applies two sublayers, each wrapped in a residual connection and layer normalisation.

Definition 12.1 (Encoder layer)**.** Let $x \in \mathrm{M}_{n\times d_{\text{model}}}(\mathbb{R})$ be the input sequence. An *encoder layer* produces

$$\begin{aligned} x' &:= \mathrm{LayerNorm}\big(x + \mathrm{MHA}(x,x,x)\big), \\ x'' &:= \mathrm{LayerNorm}\big(x' + \mathrm{FFN}(x')\big), \end{aligned}$$

where $\mathrm{MHA}(x,x,x)$ is self-attention (queries, keys, and values all from the same sequence x), and FFN is applied row-wise.

Remark 12.2 (Why exactly two sub-layers)**.** The two sub-layers serve structurally distinct roles with respect to the sequence matrix $X \in \mathrm{M}_{n\times d_{\text{model}}}(\mathbb{R})$:

Sub-layer	What it does to rows	Information flow
MHA	mixes rows (row i attends to all rows j)	across positions (the sequence dimension)
FFN	transforms each row independently via the same f	within each position (the feature dimension)

MHA is the only place where row i can "see" row $j \neq i$. The FFN then gives each position its own nonlinear transformation without exchanging information. Together the two sub-layers form one complete processing unit: first relate tokens to each other, then transform each token.

There is no mathematical necessity for exactly two; the choice $N_{\text{sub}} = 2$ is the design decision of Vaswani et al. that proved empirically effective.

Remark 12.3 (Residual connections)**.** The map $x \mapsto x + F(x)$ is a perturbation of the identity. For the Jacobian: $J(x + F(x)) = I + JF(x)$, which is invertible when $\|JF(x)\| < 1$ (by Neumann series). Residual connections ease gradient flow during training by ensuring the Jacobian of the full stack has identity as its leading term.

An encoder of depth $L = 6$ is the composition of L such layers:

$$\mathrm{Encoder} \;:=\; \mathrm{Layer}_6 \circ \cdots \circ \mathrm{Layer}_1 \;:\; \mathrm{M}_{n\times d_{\text{model}}}(\mathbb{R}) \;\to\; \mathrm{M}_{n\times d_{\text{model}}}(\mathbb{R}).$$

13. The Decoder Stack

The decoder has a different job from the encoder: given the encoder's output (a summary of the source sequence), it must *generate* a target sequence one token at a time. To do this it needs two capabilities the encoder does not:

(i) **Causal self-attention**: decoder position i may only attend to positions $j \leq i$, not future positions $j > i$ (otherwise the model could "look ahead" and trivially copy the answer).
(ii) **Cross-attention**: each decoder position must be able to read from the encoder output (the encoded source sequence).

Accordingly, each decoder layer has *three* sub-layers instead of two.

Definition 13.1 (Causal mask)**.** The *causal mask* (also called an *autoregressive mask*) of size $m \times m$ is the matrix $M \in (\mathbb{R} \cup \{-\infty\})^{m\times m}$ defined by

$$M_{ij} \;:=\; \begin{cases} 0 & i \geq j \quad \text{(position } i \text{ may attend to position } j\text{)}, \\ -\infty & i < j \quad \text{(position } i \text{ is blocked from position } j\text{)}. \end{cases}$$

Adding M to the score matrix before softmax forces future positions to receive zero attention weight:

$$P_{ij} \;=\; \frac{\exp(S_{ij} + M_{ij})}{\sum_{j'} \exp(S_{ij'} + M_{ij'})} \;=\; \begin{cases} \dfrac{\exp(S_{ij})}{\sum_{j'=1}^{i} \exp(S_{ij'})} & j \leq i, \\ 0 & j > i, \end{cases}$$

since $\exp(-\infty) = 0$. Row i of P is therefore a probability distribution supported on $\{1, \ldots, i\}$ only.

Remark 13.2 (Why π^{-1}, and why causal masking enables training)**.** During training the full target sequence is fed to the decoder simultaneously ("teacher forcing"), so without masking each position could attend to all others — including future tokens. The causal mask simulates the autoregressive inference condition: position i predicts token $i+1$ using only tokens $1, \dots, i$. At inference time the mask is not needed because generation is genuinely sequential (token $i+1$ has not been produced yet when predicting it), but it must be present during training to prevent the model from "cheating".

Definition 13.3 (Masked multi-head self-attention)**.** Let $y \in \mathrm{M}_{m \times d_{\text{model}}}(\mathbb{R})$ be the decoder's current sequence. *Masked MHA* is identical to Definition 10.1 except that the score matrix for each head ℓ is shifted by the causal mask before softmax:

$$S_\ell \;:=\; \frac{(yW_\ell^Q)(yW_\ell^K)^\top}{\sqrt{d_k}} + M \;\in\; \mathrm{M}_{m\times m}(\mathbb{R}),$$

$$\mathrm{MaskedMHA}(y) \;:=\; \big[\mathrm{softmax}_{\text{rows}}(S_1)yW_1^V \;\|\; \cdots \;\|\; \mathrm{softmax}_{\text{rows}}(S_h)yW_h^V\big]W^O \;\in\; \mathrm{M}_{m\times d_{\text{model}}}(\mathbb{R}).$$

Definition 13.4 (Cross-attention)**.** Let $y \in \mathrm{M}_{m\times d_{\text{model}}}(\mathbb{R})$ be the decoder sequence and $e \in \mathrm{M}_{n\times d_{\text{model}}}(\mathbb{R})$ be the encoder output (possibly with $m \neq n$).

Cross-attention (§3.1, called "encoder-decoder attention" in the paper) ⟨*precisely:* MHA where queries come from the decoder sequence y and keys and values come from the encoder output e⟩:

$$\mathrm{CrossMHA}(y, e) \;:=\; \mathrm{MHA}(y, e, e).$$

Expanding: for each head ℓ,

$$Q_\ell = yW_\ell^Q \in \mathrm{M}_{m\times d_k}(\mathbb{R}), \quad K_\ell = eW_\ell^K \in \mathrm{M}_{n\times d_k}(\mathbb{R}), \quad V_\ell = eW_\ell^V \in \mathrm{M}_{n\times d_v}(\mathbb{R}).$$

The score matrix $S_\ell = Q_\ell K_\ell^\top/\sqrt{d_k} \in \mathrm{M}_{m\times n}(\mathbb{R})$ has one row per decoder position and one column per encoder position. No causal mask is applied: each decoder position may attend to all n encoder positions freely.

Remark 13.5 (Cross-attention dimensions)**.** Unlike self-attention, where the score matrix is always square ($n \times n$), the cross-attention score matrix $S_\ell \in \mathrm{M}_{m\times n}(\mathbb{R})$ is rectangular when $m \neq n$ (e.g. source and target sentences of different lengths). The output still has shape $\mathrm{M}_{m\times d_{\text{model}}}(\mathbb{R})$ (same as the decoder sequence): each of the m decoder positions produces one output row.

Definition 13.6 (Decoder layer)**.** Let $y \in \mathrm{M}_{m\times d_{\text{model}}}(\mathbb{R})$ be the decoder input and $e \in \mathrm{M}_{n\times d_{\text{model}}}(\mathbb{R})$ be the (fixed) encoder output. A *decoder layer* applies three sub-layers in sequence:

$$\begin{aligned} y^{(1)} &:= \mathrm{LayerNorm}\big(y + \mathrm{MaskedMHA}(y)\big), \\ y^{(2)} &:= \mathrm{LayerNorm}\big(y^{(1)} + \mathrm{CrossMHA}(y^{(1)}, e)\big), \\ y^{(3)} &:= \mathrm{LayerNorm}\big(y^{(2)} + \mathrm{FFN}(y^{(2)})\big), \end{aligned}$$

each wrapped in a residual connection and layer normalisation.

Remark 13.7 (Why three sub-layers in the decoder vs. two in the encoder)**.**

Sub-layer	Operation	Purpose
1	$\mathrm{MaskedMHA}(y)$	decoder tokens relate to each other (causally: position i sees only $j \le i$)
2	$\mathrm{CrossMHA}(y^{(1)}, e)$	decoder tokens read from encoder output (source context)
3	$\mathrm{FFN}(y^{(2)})$	per-position nonlinear transformation

The encoder has no sub-layer 2 because there is no external context to read: it processes only the source sequence. The decoder needs sub-layer 2 to incorporate the encoded source representation.

A decoder of depth $L = 6$ is:

$$\text{Decoder}(y, e) \;:=\; \text{Layer}_6^{\text{dec}}(\,\cdot\,, e) \circ \cdots \circ \text{Layer}_1^{\text{dec}}(y, e) \;:\; \text{M}_{m\times d_{\text{model}}}(\mathbb{R}) \;\to\; \text{M}_{m\times d_{\text{model}}}(\mathbb{R}).$$

The encoder output e is computed once and passed unchanged to every decoder layer.

14. Computational Complexity of Self-Attention

Definition 14.1 (FLOPs and sequential operations)**.**

- **FLOPs** (floating-point operations): the total number of arithmetic operations (multiplications, additions, etc.) required for one forward pass through the layer. This measures *total arithmetic work*, independent of whether that work can be parallelised. On a single processor, FLOPs determine running time; on many processors, they set a lower bound even if parallelism is exploited.
- **Sequential operations**: the length of the *critical path* — the longest chain of computations in which each step depends on the output of the previous. Even with *infinitely many* parallel processors, you still need at least this many sequential steps.
  - $O(1)$ sequential ops: the computation has no inherent ordering; all arithmetic can in principle proceed in a constant number of parallel rounds regardless of n.
  - $O(n)$ sequential ops: there is a chain of n steps that must be executed one after another, no matter how many processors are available.

The two metrics are independent: a computation can have large FLOPs but small sequential depth (lots of parallelisable work), or small FLOPs but large sequential depth (an inherently serial computation).

Proposition 14.2 (Complexity table)**.** *For a sequence of length n and per-position dimension d:*

Layer type	*FLOPs*	*Sequential ops*
Self-attention	$O(n^2 d)$	$O(1)$
Recurrent (RNN/LSTM)	$O(nd^2)$	$O(n)$
Convolution (kernel width k)	$O(knd^2)$	$O(1)$

Remark 14.3 (Derivation of each row)**.**

Self-attention ($O(n^2 d)$ FLOPs, $O(1)$ sequential). One forward pass through $\text{Att}(Q, K, V)$ consists of three steps:

(i) $S = QK^\top/\sqrt{d_k}$: matrix product of $Q \in \text{M}_{n\times d_k}(\mathbb{R})$ with $K^\top \in \text{M}_{d_k\times n}(\mathbb{R})$. Entry (i, j) of S is the dot product $q_i \cdot k_j/\sqrt{d_k}$, requiring d_k multiplications and $d_k - 1$ additions. With n^2 entries: $O(n^2 d_k)$ operations.

(ii) $P = \text{softmax}_{\text{rows}}(S)$: for each of the n rows, one pass over n entries to compute exp and sum: $O(n)$ per row, $O(n^2)$ total.

(iii) $O = PV$: matrix product of $P \in \text{M}_{n\times n}(\mathbb{R})$ with $V \in \text{M}_{n\times d_v}(\mathbb{R})$. Each of the $n \cdot d_v$ entries is a length-n dot product: $O(n^2 d_v)$.

Total: $O(n^2 d_k + n^2 + n^2 d_v) = O(n^2 d)$ (setting $d_k, d_v \asymp d$).

Sequential depth: $O(1)$. Every entry of S, P, O can be computed independently from the others (given Q, K, V). The three steps above are each a "round" of parallel computation; the whole forward pass completes in $O(1)$ sequential rounds regardless of n.

Recurrent layer ($O(nd^2)$ FLOPs, $O(n)$ sequential). An RNN computes a hidden state sequence $h_1, \ldots, h_n \in \mathbb{R}^d$ via

$$h_t \;=\; \phi(W_h\, h_{t-1} + W_x\, x_t + b), \qquad W_h, W_x \in \text{M}_{d\times d}(\mathbb{R}),$$

where $x_t \in \mathbb{R}^d$ is the t-th input and ϕ is applied componentwise. Each step requires two matrix-vector products in $\mathbb{R}^d$: $O(d^2)$ operations. Over n steps: $O(nd^2)$ total.

Sequential depth: $O(n)$. Step t requires h_{t-1}, which requires h_{t-2}, ..., which requires h_1. This is a chain of length n; no step can begin until the previous has finished. No amount of parallelism can shorten this chain.

Convolutional layer ($O(knd^2)$ FLOPs, $O(1)$ sequential). A 1-D temporal convolution with kernel width k computes, for each output position $t \in \{1, \ldots, n\}$:

$$O_t \;=\; \sum_{j=0}^{k-1} W_j \, x_{t-j}, \qquad W_j \in \mathrm{M}_{d\times d}(\mathbb{R}).$$

Each of the k terms is a matrix-vector product in $\mathbb{R}^d$: $O(d^2)$ per term, $O(kd^2)$ per output position, $O(knd^2)$ over all n positions.

Sequential depth: $O(1)$. All n output positions are computed from the *input* $x_1, \ldots, x_n$ only (not from each other), so they are fully independent and can be computed in parallel.

When is each regime cheaper?

- $n \ll d$: $n^2 d \ll nd^2$, so attention is cheaper than recurrence.
- $n \gg d$: $n^2 d \gg nd^2$, so recurrence is cheaper per-position (but incurs $O(n)$ sequential depth).
- Convolution vs. attention: attention wins for short sequences ($n < d$); convolution wins for local patterns where small k suffices.

FlashAttention (Papers III–IV) does *not* reduce FLOPs: the arithmetic work is still $O(n^2 d)$ and time complexity in the sequential sense is still quadratic in n. What it reduces is *space* and *memory traffic*:

- Standard attention stores the full score matrix $S \in \mathrm{M}_{n\times n}(\mathbb{R})$ and attention weight matrix $P \in \mathrm{M}_{n\times n}(\mathbb{R})$ in HBM (GPU main memory), requiring $O(n^2)$ space (quadratic in n) and $O(n^2 + nd)$ HBM read/write traffic.
- FlashAttention tiles Q, K, V into blocks that fit in SRAM (fast on-chip cache), computes and discards S and P within SRAM without ever writing them to HBM. Only $Q, K, V, O \in \mathrm{M}_{n\times d}(\mathbb{R})$ (each $O(nd)$) touch HBM. Total HBM traffic: $O(nd)$ — *linear* in n for fixed d.

Summary: **FLOPs $O(n^2 d)$ unchanged (time still quadratic in n); HBM memory $O(n^2) \to O(n)$ (space becomes linear in n, with d fixed).**

15. The Transformer as Commutative Diagrams

Notation. Throughout this section, fix $d := d_{\text{model}}$.

$x \in \mathrm{M}_{n\times d}(\mathbb{R})$**:** is the *full sequence matrix*: an $n \times d$ real matrix whose *rows* are indexed by position and whose *columns* are indexed by embedding coordinate. Row i of x is the embedding of token (position) i.

$x_{i,\cdot} \in \mathbb{R}^d$**:** is the *$i$-th row* of x; it is a *row vector* of length d, i.e. an element of $\mathbb{R}^{1\times d}$ (or equivalently $\mathbb{R}^d$ when we suppress the row/column distinction). It encodes a single token's state.

$[\cdot]_{i,\cdot} : \mathrm{M}_{n\times d}(\mathbb{R}) \to \mathbb{R}^d$**:** denotes the row-extraction map $x \mapsto x_{i,\cdot}$.

Diagram structure. Each diagram below has *two horizontal chains* connected by *vertical arrows*:

(i) **Sequence level (top chain, $\to$)**: the full $n \times d$ matrix at each stage, with its membership $\in \mathrm{M}_{n\times d}(\mathbb{R})$ written inline.

(ii) **Token level (bottom chain, $\mapsto$)**: the i-th row at each stage, with its membership $\in \mathbb{R}^d$ written inline.

(iii) **Vertical arrows** labeled $[\cdot]_{i,\cdot}$: the row-extraction maps connecting the two chains.

Each square in the diagram *genuinely commutes*: "apply the sequence-level map, then extract row i" gives the same result as "extract row i, then apply the token-level map":

$$[\cdot]_{i,\cdot} \circ A \;=\; [A(\,\cdot\,)]_{i,\cdot}.$$

Global vs. local at the token level. The token-level arrow labels are chosen to make the dependence explicit:

- A label of the form $[A(x)]_{i,\cdot}$ — with the *full matrix* x appearing inside $A(\,\cdot\,)$ — signals a **global** operation: the image of row i depends on ALL rows of the input. The map is NOT a self-contained function $\mathbb{R}^d \to \mathbb{R}^d$; it is row i of a function $\mathrm{M}_{n\times d}(\mathbb{R}) \to \mathrm{M}_{n\times d}(\mathbb{R})$. (MHA, MaskedMHA, CrossMHA.)
- A label of the form $A(x_{i,\cdot})$ — with *only the row vector* $x_{i,\cdot}$ appearing — signals a **local** (position-wise) operation: it IS a self-contained function $\mathbb{R}^d \to \mathbb{R}^d$. (FFN, LayerNorm.)

Operation	Token-level arrow label	Kind
MHA	$[A_1(x)]_{i,\cdot}$	global (all j)
MaskedMHA	$[B_1(y)]_{i,\cdot}$	global ($j \le i$ only)
CrossMHA	$[B_2(y')]_{i,\cdot}$	global (all encoder positions $e_{k,\cdot}$)
FFN	$A_2(x'_{i,\cdot})$	local (only row i)

Encoder layer. Define the two sub-layer maps:

$$\begin{aligned} A_1 &:= \mathrm{LayerNorm}\big(\,\cdot\, + \mathrm{MHA}(\,\cdot\,,\cdot\,,\cdot\,)\big) : \mathrm{M}_{n\times d}(\mathbb{R}) \to \mathrm{M}_{n\times d}(\mathbb{R}), \\ A_2 &:= \mathrm{LayerNorm}\big(\,\cdot\, + \mathrm{FFN}(\,\cdot\,)\big) : \mathrm{M}_{n\times d}(\mathbb{R}) \to \mathrm{M}_{n\times d}(\mathbb{R}). \end{aligned}$$

One encoder layer is $\mathrm{Layer}^{\mathrm{enc}} = A_2 \circ A_1$.

(1)

$$\begin{array}{ccccc} x \in \mathrm{M}_{n\times d}(\mathbb{R}) & \xrightarrow{\quad A_1 \quad} & x' \in \mathrm{M}_{n\times d}(\mathbb{R}) & \xrightarrow{\quad A_2 \quad} & x'' \in \mathrm{M}_{n\times d}(\mathbb{R}) \\ \big\downarrow{\scriptstyle [\cdot]_{i,\cdot}} & & \big\downarrow{\scriptstyle [\cdot]_{i,\cdot}} & & \big\downarrow{\scriptstyle [\cdot]_{i,\cdot}} \\ x_{i,\cdot} \in \mathbb{R}^d & \xmapsto[[A_1(x)]_{i,\cdot}]{} & x'_{i,\cdot} \in \mathbb{R}^d & \xmapsto[A_2(x'_{i,\cdot})]{} & x''_{i,\cdot} \in \mathbb{R}^d \end{array}$$

The full encoder of depth $L = 6$ is the composition

$$x^{(0)} \xrightarrow{\mathrm{Layer}_1^{\mathrm{enc}}} x^{(1)} \xrightarrow{\mathrm{Layer}_2^{\mathrm{enc}}} \cdots \xrightarrow{\mathrm{Layer}_L^{\mathrm{enc}}} e = x^{(L)}, \qquad x^{(0)}, x^{(1)}, \ldots, e \in \mathrm{M}_{n\times d}(\mathbb{R}),$$

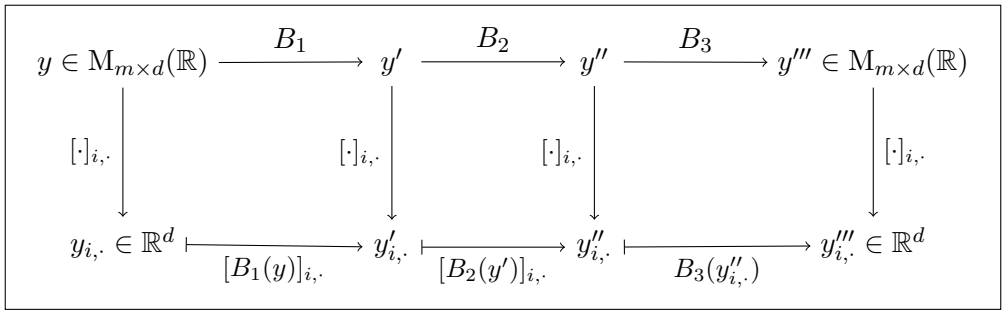
where $\mathrm{Layer}_\ell^{\mathrm{enc}} = A_2^{(\ell)} \circ A_1^{(\ell)}$ with independent parameters per layer. The output $e = x^{(L)} \in \mathrm{M}_{n\times d}(\mathbb{R})$ is the *encoder memory*, passed (unchanged) to every decoder layer.

Decoder layer. Define the three sub-layer maps (with fixed encoder memory $e \in \mathrm{M}_{n\times d}(\mathbb{R})$):

$$\begin{aligned} B_1 &:= \mathrm{LayerNorm}\big(\,\cdot\, + \mathrm{MaskedMHA}(\,\cdot\,)\big) : \mathrm{M}_{m\times d}(\mathbb{R}) \to \mathrm{M}_{m\times d}(\mathbb{R}), \\ B_2 &:= \mathrm{LayerNorm}\big(\,\cdot\, + \mathrm{CrossMHA}(\,\cdot\,, e)\big) : \mathrm{M}_{m\times d}(\mathbb{R}) \to \mathrm{M}_{m\times d}(\mathbb{R}), \\ B_3 &:= \mathrm{LayerNorm}\big(\,\cdot\, + \mathrm{FFN}(\,\cdot\,)\big) : \mathrm{M}_{m\times d}(\mathbb{R}) \to \mathrm{M}_{m\times d}(\mathbb{R}). \end{aligned}$$

One decoder layer is $\mathrm{Layer}^{\mathrm{dec}} = B_3 \circ B_2 \circ B_1 : \mathrm{M}_{m\times d}(\mathbb{R}) \to \mathrm{M}_{m\times d}(\mathbb{R})$.

(2)



The complete Transformer. Source input → encoder memory → decoder output → vocabulary distribution:

$$\underbrace{x^{(0)}_{\mathrm{src}} \in \mathrm{M}_{n\times d}(\mathbb{R})}_{\text{source embeddings+PE}} \xrightarrow{\text{Encoder}} e \in \mathrm{M}_{n\times d}(\mathbb{R})$$

$$\underbrace{y^{(0)}_{\mathrm{tgt}} \in \mathrm{M}_{m\times d}(\mathbb{R})}_{\text{target embeddings+PE}} \xrightarrow{\mathrm{Decoder}(\,\cdot\,,e)} y^{(L)} \in \mathrm{M}_{m\times d}(\mathbb{R}) \xrightarrow{\mathrm{softmax}_{\mathrm{rows}}(\,\cdot\, W_{\mathrm{out}})} \hat{P} \in \mathrm{M}_{m\times|\mathcal{V}|}(\mathbb{R}),$$

where $W_{\mathrm{out}} \in \mathrm{M}_{d\times|\mathcal{V}|}(\mathbb{R})$ and each row $\hat{P}_{i,\cdot} \in \mathrm{int}(\Delta^{|\mathcal{V}|-1})$ is the predicted probability distribution over the vocabulary at target position i.

16. Summary of Paper I

The mathematical content of "Attention Is All You Need" can be summarised as:

(1) **Attention** (Definition 6.2) is a smooth map $\mathrm{Att} : \mathrm{M}_{n\times d_k}(\mathbb{R})^2 \times \mathrm{M}_{n\times d_v}(\mathbb{R}) \to \mathrm{M}_{n\times d_v}(\mathbb{R})$ that applies row-wise softmax to the rescaled Gram matrix $QK^\top/\sqrt{d_k}$, then right-multiplies by V.
(2) **Softmax** (Definition 5.2) is the gradient of the log-partition function lse, and descends to a diffeomorphism $\mathbb{R}^n/\mathbb{R}\mathbf{1} \xrightarrow{\sim} \mathrm{int}(\Delta^{n-1})$ (Proposition 5.5).
(3) **Scaling by** $1/\sqrt{d_k}$ normalises the variance of dot products to 1 (Proposition 7.1), preventing softmax saturation.
(4) **Attention is S_n-equivariant** (Proposition 8.3): it treats the sequence as an unordered set. Positional encoding (Definition 9.1) breaks this symmetry, with the rotation property (Proposition 9.2) enabling relative-position learning.
(5) **Multi-head attention** (Definition 10.1) decomposes the attention map into h parallel maps in lower-dimensional projected subspaces, concatenated and projected back.
(6) The **encoder stack** (Section 12) is a depth-L composition of self-attention + FFN sublayers, each wrapped in residual connections and layer normalisation.

What is missing from the paper's analysis: The paper does not analyse the softmax geometry, the permutation equivariance, or the information-geometric structure of attention. These are implicit in the construction but not made explicit. Papers II–IV will build directly on Points (1)–(2): they exploit Proposition 5.6 and the merge formula for lse to compute attention without materialising S or P.

Part 3. Paper II: Online Normalizer Calculation for Softmax

17. The Safe-Softmax Statistics

Recall from Definition 5.2:

$$\mathrm{softmax}(x)_i = e^{x_i - \mathrm{lse}(x)}, \qquad \mathrm{lse}(x) = \log \textstyle\sum_{j=1}^{n} e^{x_j}, \qquad x \in \mathbb{R}^n.$$

The naïve evaluation $e^{x_i}/\sum_j e^{x_j}$ overflows in finite precision when $\max_j x_j$ is large. Proposition 5.6 (translation invariance) guarantees $\mathrm{softmax}(x) = \mathrm{softmax}(x - m\mathbf{1})$ for any $m \in \mathbb{R}$; choosing $m = \max_j x_j$ bounds every exponent in $(-\infty, 0]$, preventing overflow.

Definition 17.1 (Safe-softmax statistics)**.** For $x \in \mathbb{R}^n$, define the *safe-softmax statistics*

$$m(x) \;:=\; \max_{i=1}^{n} x_i \;\in\; \mathbb{R},$$
$$\ell(x) \;:=\; \sum_{i=1}^{n} e^{x_i - m(x)} \;\in\; \mathbb{R}_{>0}.$$

Note $\ell(x) \geq 1$ (the term at $i = \arg\max x_i$ contributes $e^0 = 1$), and $e^{x_i - m(x)} \in (0, 1]$ for all i, so both quantities are safe for IEEE 754 arithmetic.

Remark 17.2 (Factored form of lse, and recovery of softmax)**.**

$$\mathrm{lse}(x) \;=\; m(x) + \log \ell(x).$$

Proof. $m(x) + \log \ell(x) = m(x) + \log\big(e^{-m(x)} \sum_i e^{x_i}\big) = \log \sum_i e^{x_i} = \mathrm{lse}(x)$. □

Consequently $\mathrm{softmax}(x)_i = e^{x_i - m(x)}/\ell(x)$. The pair $(m(x), \ell(x))$ is a *numerically stable factorisation* of $\mathrm{lse}(x)$: no overflow can occur in either factor.

18. The Merge Monoid

The central algebraic content of Milakov & Gimelshein (2018) is that the safe-softmax statistics of a disjoint union equal the *merge* of the partial statistics, and that this merge operation gives a commutative monoid.

Definition 18.1 (Monoid)**.** A *monoid* is a triple $(M, \cdot, e)$ where M is a set, $\cdot : M \times M \to M$ is a binary operation, and $e \in M$ is a distinguished element, satisfying:

(i) **Associativity**: $(a \cdot b) \cdot c = a \cdot (b \cdot c)$ for all $a, b, c \in M$.
(ii) **Identity**: $e \cdot a = a \cdot e = a$ for all $a \in M$.

A monoid is *commutative* (or *abelian*) if additionally $a \cdot b = b \cdot a$ for all $a, b \in M$.

Remark 18.2. A monoid is a group without the requirement of inverses. Familiar examples: $(\mathbb{N}, +, 0)$, $(\mathbb{R}, \cdot, 1)$, $(M_{n\times n}(\mathbb{R}), \cdot, I_n)$ (the last is non-commutative for $n \geq 2$). The absence of inverses is not a deficiency here: the merge operation $\oplus$ below need not be invertible for the tiling algorithm to work.

Definition 18.3 (Merge operation)**.** Write $\overline{\mathbb{R}} := \mathbb{R} \cup \{-\infty\}$ and adopt the convention $e^{-\infty} := 0$. Set $S := \overline{\mathbb{R}} \times \mathbb{R}_{\geq 0}$ and define

$$(m_A, \ell_A) \oplus (m_B, \ell_B) \;:=\; \Big(\max(m_A, m_B),\; e^{m_A - \max(m_A, m_B)}\, \ell_A + e^{m_B - \max(m_A, m_B)}\, \ell_B\Big).$$

Here (m_A, ℓ_A) and (m_B, ℓ_B) are arbitrary elements of S; the subscripts A and B are suggestive labels — anticipating Proposition 18.5 below, where they will be instantiated as the safe-softmax statistics $(m(x_A), \ell(x_A))$ and $(m(x_B), \ell(x_B))$ of two disjoint subsequences (Definition 17.1).

Proposition 18.4 ($(S, \oplus, (-\infty, 0))$ is a commutative monoid)**.** *The operation $\oplus$ is commutative and associative, with identity $(-\infty, 0)$.*

Proof. Write $m_{XY} := \max(m_X, m_Y)$ and $m_{XYZ} := \max(m_X, m_Y, m_Z)$.

Identity. $\max(m, -\infty) = m$ and $e^{-\infty} = 0$, so $(m, \ell) \oplus (-\infty, 0) = (m,\ e^{m-m}\ell + 0) = (m, \ell)$.

Commutativity. Both max and addition of reals are commutative.

Associativity. The first $\oplus$ gives the pair $(m_{AB},\ e^{m_A - m_{AB}}\ell_A + e^{m_B - m_{AB}}\ell_B)$. Applying $\oplus(m_C, \ell_C)$ and using the exponent identity $e^{m_{AB} - m_{ABC}} \cdot e^{m_X - m_{AB}} = e^{m_X - m_{ABC}}$:

$$\big((m_A, \ell_A) \oplus (m_B, \ell_B)\big) \oplus (m_C, \ell_C) \;=\; \Big(m_{ABC},\ e^{m_A - m_{ABC}}\ell_A + e^{m_B - m_{ABC}}\ell_B + e^{m_C - m_{ABC}}\ell_C\Big).$$

The right-bracketed product $(m_A, \ell_A) \oplus \big((m_B, \ell_B) \oplus (m_C, \ell_C)\big)$ yields the same triple by the same identity. □

Proposition 18.5 (Statistics of a disjoint union)**.** *Let $A, B \subseteq \{1, \dots, n\}$ be disjoint, $x \in \mathbb{R}^n$, and write $x_A := (x_i)_{i \in A}$ for the subsequence indexed by A. Then*

$$(m(x_A), \ell(x_A)) \oplus (m(x_B), \ell(x_B)) \;=\; (m(x_{A \cup B}),\ \ell(x_{A \cup B})).$$

Proof. Set $m := \max(m(x_A), m(x_B)) = \max_{k \in A \cup B} x_k = m(x_{A \cup B})$. Then

$$\begin{aligned} e^{m(x_A) - m}\,\ell(x_A) + e^{m(x_B) - m}\,\ell(x_B) &= e^{m(x_A) - m} \sum_{i \in A} e^{x_i - m(x_A)} + e^{m(x_B) - m} \sum_{j \in B} e^{x_j - m(x_B)} \\ &= \sum_{i \in A} e^{x_i - m} + \sum_{j \in B} e^{x_j - m} = \sum_{k \in A \cup B} e^{x_k - m} = \ell(x_{A \cup B}). \end{aligned}$$

□

19. The Online Algorithm as a Left Fold

What we are trying to do. Given $x \in \mathbb{R}^n$, we want to compute $\mathrm{softmax}(x)$, but we cannot afford to hold all of x in fast memory simultaneously. Instead, we process x one element (or one *tile* of consecutive elements) at a time, maintaining a compact running state that is updated with each new piece of data and discarded nothing. The monoid $(S, \oplus, (-\infty, 0))$ is precisely the algebraic structure that makes this possible: because $\oplus$ is associative, any sequential or parallel order of combining partial statistics gives the same global result.

What is a left fold? Given a monoid $(M, \cdot, e)$ and a sequence $(a_1, \dots, a_n) \in M^n$, the *sequential left fold* is the iterated binary product starting from the identity, processing elements left to right:

$$\mathrm{foldl}(a_1, \dots, a_n) \;:=\; \big(\cdots\big((e \cdot a_1) \cdot a_2\big)\cdots\big) \cdot a_n.$$

Equivalently, initialise an accumulator $s \leftarrow e$, then for each k update $s \leftarrow s \cdot a_k$. Associativity guarantees that the result equals $e \cdot a_1 \cdot a_2 \cdots a_n$ regardless of how the parentheses are placed, so the left fold is one concrete realisation of the monoid product of the entire sequence.

Example 19.1 (Left fold in $(S, \oplus, (-\infty, 0))$ for $n = 1, 2, 3$)**.** Take $x = (2, 5, 1) \in \mathbb{R}^3$. Each scalar x_k is embedded as the pair $(x_k, 1) \in S$, and the left fold accumulates these one at a time starting from the identity $e = (-\infty, 0)$.

$n = 1$, sequence (2):

$$\begin{aligned} s_0 &= (-\infty,\ 0), \\ s_1 &= s_0 \oplus (2, 1) = \big(\max(-\infty, 2),\ e^{-\infty} \cdot 0 + e^{2-2} \cdot 1\big) = (2,\ 1). \end{aligned}$$

Check: $m(2) = 2$ and $\ell(2) = e^{2-2} = 1$.

$n = 2$, sequence $(2, 5)$:

$$s_2 = s_1 \oplus (5, 1) = \big(\max(2, 5),\ e^{2-5} \cdot 1 + e^{5-5} \cdot 1\big) = (5,\ e^{-3} + 1).$$

Check: $m(2, 5) = 5$ and $\ell(2, 5) = e^{2-5} + e^{5-5} = e^{-3} + 1$.

$n = 3$, sequence $(2, 5, 1)$:

$$s_3 = s_2 \oplus (1, 1) = \big(\max(5, 1),\ e^{5-5}(1 + e^{-3}) + e^{1-5} \cdot 1\big) = (5,\ 1 + e^{-3} + e^{-4}).$$

Check: $m(2,5,1) = 5$ and $\ell(2,5,1) = e^{2-5} + e^{5-5} + e^{1-5} = e^{-3} + 1 + e^{-4}$.

In each step the accumulator s_k holds the exact safe-softmax statistics (m, ℓ) of the prefix $(x_1, \ldots, x_k)$ seen so far; no past value needs to be retained.

Remark 19.2 (The monoid acts on the accumulator state)**.** The update rule $s \leftarrow s \oplus (x_k, 1)$ is an instance of a *left monoid action*: the monoid $(S, \oplus, e)$ acts on the set S by left multiplication, $s \mapsto s \oplus t$ for a fixed $t \in S$. Each new data point $(x_k, 1)$ is an element of S that *acts* on the current state to produce the next state. Because $\oplus$ is associative, the order in which the actions are applied does not matter for the final result — only the multiset of elements acted upon. The left fold is the canonical way to realise this iterated action sequentially.

Corollary 19.3 (Fold representation)**.** *For any single element $x_k \in \mathbb{R}$: $(m(\{x_k\}), \ell(\{x_k\})) = (x_k, 1)$. By Proposition 18.5 and associativity of $\oplus$:*

$$(m(x),\, \ell(x)) \;=\; \bigoplus_{k=1}^{n} (x_k, 1) \;=\; (-\infty, 0) \oplus (x_1, 1) \oplus (x_2, 1) \oplus \cdots \oplus (x_n, 1),$$

computable as the following sequential left fold:

Online softmax statistics *(single pass over x)*

Initialise $(m, \ell) \leftarrow (-\infty, 0)$.

For $k = 1, \ldots, n$*:*

$m' \leftarrow \max(m,\, x_k)$

$\ell' \leftarrow e^{m - m'}\, \ell \;+\; e^{x_k - m'}$

$(m, \ell) \leftarrow (m', \ell')$

After the pass: $\mathrm{softmax}(x)_i = e^{x_i - m}/\ell$ *for* $i = 1, \ldots, n$.

Remark 19.4 (Parallelism from associativity)**.** Because $\oplus$ is associative, the input x may be partitioned into *any number of disjoint tiles* $x_{A_1}, \ldots, x_{A_T}$ (not necessarily contiguous, not necessarily processed in order), their partial statistics merged in any binary tree of $\oplus$ applications, and the result equals $(m(x), \ell(x))$. Commutativity removes even the ordering constraint between tiles. This is the algebraic prerequisite for the GPU tiling strategy of Paper III.

Remark 19.5 (Two passes vs. one)**.** The procedure above requires a *second pass* through x to evaluate $e^{x_i - m}/\ell$ for each i. In the attention context (Paper III), $x = s$ is a score row of length n — too large to hold in on-chip SRAM — so this second pass would require re-reading s from HBM. The *attention output accumulator* (Section 20) eliminates this second pass by propagating the partial weighted sum of V alongside (m, ℓ).

Remark 19.6 (What is to be gained: the chain of ideas)**.** The three sections of Paper II form a single chain of reasoning:

(1) **Numerical stability** (Section 17). The naïve evaluation of softmax overflows. The pair $(m(x), \ell(x))$ is a stable, lossless representation of the same information as $\mathrm{lse}(x)$, with no intermediate value exceeding 1 in magnitude.

(2) **Algebraic structure** (Section 18). The pair (m, ℓ) is not just a computational trick: it is an element of a *commutative monoid* $(S, \oplus, (-\infty, 0))$. This means partial statistics from *different tiles* of x can be combined in any order without losing correctness. Neither commutativity nor associativity is obvious from the formula; both are verified in Proposition 18.4.

(3) **Single-pass algorithm** (this section). Monoid structure implies a left fold: process x sequentially, keeping only a two-number running state (m, ℓ), and recover $\mathrm{softmax}(x)$ at the end. No random access to earlier elements is required. In the GPU setting, this means each tile of x is read from HBM *once*, processed in SRAM, and discarded. The $n \times n$ attention

score matrix S — which standard attention materialises entirely in HBM — never needs to exist as a stored object.

(4) **Setup for Paper III** (Section 20). Extending the monoid to triples $(m, \ell, \tilde{o})$ carries the partial attention *output* alongside the statistics, allowing the full computation $\mathrm{Att}(q, K, V) = \mathrm{softmax}(s)\, V$ to be done tile by tile. This is exactly the FlashAttention inner loop.

The payoff: FlashAttention achieves the *same numerical output* as standard attention while reducing HBM memory traffic from $O(n^2)$ to $O(n)$ (in sequence length n, with head dimension d fixed).

20. The Attention Output Accumulator

Fix a single query row $q \in \mathbb{R}^{d_k}$ and recall that $\mathrm{Att}(q, K, V) = \mathrm{softmax}(s)\, V \in \mathbb{R}^{d_v}$ where $s_j := qK_{j,\cdot}^{\top}/\sqrt{d_k}$.

Definition 20.1 (Attention accumulator)**.** For a subset $A \subseteq \{1, \dots, n\}$ of key–value positions, the *attention accumulator* is the triple

$$\alpha(A) \;:=\; \big(m(s_A),\ \ell(s_A),\ \tilde{o}(A)\big) \;\in\; \overline{\mathbb{R}} \times \mathbb{R}_{\geq 0} \times \mathbb{R}^{d_v},$$

where $s_A := (s_j)_{j \in A}$ and the *unnormalised partial output* is

$$\tilde{o}(A) \;:=\; \sum_{j \in A} e^{s_j - m(s_A)}\, V_{j,\cdot} \;=\; \ell(s_A) \cdot \mathrm{softmax}(s_A)\, V_A \;\in\; \mathbb{R}^{d_v}.$$

The *normalised partial output* is $o(A) := \tilde{o}(A)/\ell(s_A) \in \mathbb{R}^{d_v}$.

Proposition 20.2 (Merge of attention accumulators)**.** *For disjoint $A, B \subseteq \{1, \dots, n\}$, define the merge*

$$\begin{aligned} m &:= \max(m(s_A), m(s_B)), \\ \ell_{A \cup B} &:= e^{m(s_A) - m}\, \ell(s_A) + e^{m(s_B) - m}\, \ell(s_B), \\ \tilde{o}(A \cup B) &:= e^{m(s_A) - m}\, \tilde{o}(A) + e^{m(s_B) - m}\, \tilde{o}(B). \end{aligned}$$

Then $\alpha(A \cup B) = (m, \ell_{A \cup B}, \tilde{o}(A \cup B))$. In particular, the normalised output satisfies

$$o(A \cup B) \;=\; \underbrace{\frac{e^{m(s_A) - m}\, \ell(s_A)}{\ell_{A \cup B}}}_{\lambda_A}\, o(A) \;+\; \underbrace{\frac{e^{m(s_B) - m}\, \ell(s_B)}{\ell_{A \cup B}}}_{\lambda_B}\, o(B), \qquad \lambda_A + \lambda_B = 1,\ \lambda_A, \lambda_B > 0.$$

That is, $o(A \cup B)$ is a convex combination *of $o(A)$ and $o(B)$.*

Proof. The first two components follow from Proposition 18.5. For $\tilde{o}$:

$$\begin{aligned} \tilde{o}(A \cup B) &= \sum_{k \in A \cup B} e^{s_k - m}\, V_{k,\cdot} \\ &= \sum_{i \in A} e^{(s_i - m(s_A)) + (m(s_A) - m)}\, V_{i,\cdot} + \sum_{j \in B} e^{(s_j - m(s_B)) + (m(s_B) - m)}\, V_{j,\cdot} \\ &= e^{m(s_A) - m} \sum_{i \in A} e^{s_i - m(s_A)}\, V_{i,\cdot} + e^{m(s_B) - m} \sum_{j \in B} e^{s_j - m(s_B)}\, V_{j,\cdot} \\ &= e^{m(s_A) - m}\, \tilde{o}(A) + e^{m(s_B) - m}\, \tilde{o}(B). \end{aligned}$$

Dividing by $\ell_{A \cup B}$ yields the convex combination for $o(A \cup B)$, with $\lambda_A + \lambda_B = \ell_{A \cup B}/\ell_{A \cup B} = 1$. □

Remark 20.3 (Second commutative monoid)**.** The triples $(m, \ell, \tilde{o})$ form a commutative monoid under the componentwise extension of $\oplus$:

$$(m_A, \ell_A, \tilde{o}_A) \oplus (m_B, \ell_B, \tilde{o}_B) \;:=\; \Big(\max(m_A, m_B),\ e^{m_A - m} \ell_A + e^{m_B - m} \ell_B,\ e^{m_A - m} \tilde{o}_A + e^{m_B - m} \tilde{o}_B\Big)$$

with identity $(-\infty, 0, \mathbf{0})$. Associativity follows by the same telescoping as Proposition 18.4: both ℓ and $\tilde{o}$ merge *linearly* (after rescaling by $e^{m_X - m}$), so the three-component version inherits associativity componentwise.

The FlashAttention algorithm (Paper III) is exactly the sequential left fold

$$\bigoplus_{t=1}^{T} \alpha(A_t) \;=\; \alpha(\{1, \ldots, n\}),$$

where $A_1, \ldots, A_T$ tile the key–value index set, each tile A_t is computed in SRAM, and the merge accumulates $(m, \ell, \tilde{o})$ without ever writing the $n \times n$ score matrix S to HBM.

Part 4. Paper III: FlashAttention — Fast and Memory-Efficient Exact Attention with IO-Awareness

21. The GPU Memory Hierarchy

Modern GPU computation involves two qualitatively different storage levels.

Remark 21.1 (CUDA programming terminology)**.** GPU programmers working in CUDA use a parallel vocabulary for the same hardware. The correspondence is:

FlashAttention paper term	CUDA / GPU-architecture term
HBM (high-bandwidth memory)	global memory
SRAM (on-chip static RAM)	shared memory (per thread block)
IO operation	global memory read/write

Global memory is the CUDA programmer's name for the large, slow, off-chip DRAM that all threads can read and write; the standard speed ordering is

$$\text{registers} \;>\; \text{shared memory} \;\gg\; \text{global memory},$$

where $\gg$ indicates that the gap between shared and global memory bandwidth is the dominant performance bottleneck.

The name *HBM* (High Bandwidth Memory) refers to the specific physical DRAM technology used in data-centre GPUs: the A100 and H100 use HBM2e and HBM3, respectively. Consumer GPUs (e.g. RTX 3060, 4090) use GDDR6X instead, but both are "global memory" in CUDA's sense. FlashAttention uses the hardware term because its IO-complexity analysis targets the A100; a CUDA programmer should read "HBM" as "global memory" throughout.

Definition 21.2 (Two-level memory model)**.** Fix two capacity parameters $M_H \gg M > 0$ (measured in number of floating-point elements).

- **HBM** (global memory): capacity M_H, bandwidth f_H. All inputs and outputs of the computation reside here initially and finally.
- **SRAM** (shared memory, on-chip): capacity M, bandwidth $f_S \gg f_H$. Arithmetic units can only operate on data in SRAM.

An *IO operation* is a transfer of one element between HBM and SRAM (a load HBM $\to$ SRAM or a store SRAM $\to$ HBM). The *IO complexity* of an algorithm is the total number of IO operations it performs.

Definition 21.3 (Arithmetic intensity)**.** The *arithmetic intensity* of an algorithm is the ratio

$$I \;:=\; \frac{\text{floating-point operations (FLOPs)}}{\text{bytes transferred between HBM and SRAM}}.$$

An algorithm is *compute-bound* if increasing FLOPs/s would reduce its runtime; it is *memory-bandwidth-bound* (or *IO-bound*) if increasing HBM bandwidth would reduce its runtime. Maximising

arithmetic intensity — doing as much arithmetic as possible per byte moved — is the central goal of GPU kernel optimisation.

Remark 21.4 (Why IO dominates for attention)**.** On an NVIDIA A100 GPU: $M_H = 40\,\text{GB}$, $f_H = 1.5\,\text{TB/s}$; $M = 192\,\text{KB}$ per streaming multiprocessor (SM), $f_S \approx 19\,\text{TB/s}$. The Tensor Core units deliver $\approx 312\,\text{TFLOP/s}$ of arithmetic throughput.

For standard attention at $n = 4096$, $d_k = 64$: the score matrix $S \in \mathrm{M}_{4096\times 4096}(\mathbb{R})$ occupies $4096^2 \times 4\,\text{B} = 64\,\text{MB}$. Reading S from HBM takes $64\,\text{MB}/1.5\,\text{TB/s} \approx 43\,\mu\text{s}$; computing $S = QK^\top/\sqrt{d_k}$ via Tensor Cores (once Q and K are in SRAM) takes $\approx 2\times 4096^2 \times 64\,\text{FLOPs}/312\,\text{TFLOP/s} \approx 0.7\,\mu\text{s}$.

Standard attention is therefore memory-bandwidth-bound by a factor of $\approx 60\times$: Tensor Cores sit mostly idle waiting for data to arrive from HBM. Arithmetic is cheap; moving data is the bottleneck.

22. IO Complexity of Standard Attention

Standard attention materialises both intermediate matrices S and P in HBM because they are $n \times n$ and do not fit in SRAM for large n.

Proposition 22.1 (IO complexity of standard attention)**.** *Let $Q, K \in \mathrm{M}_{n\times d_k}(\mathbb{R})$ and $V \in \mathrm{M}_{n\times d_v}(\mathbb{R})$ with $n \gg d_k, d_v$. Computing $O = \mathrm{Att}(Q, K, V)$ by the naïve three-step procedure*

$$\begin{aligned} S &= QK^\top/\sqrt{d_k} \;\in\; \mathrm{M}_{n\times n}(\mathbb{R}), \\ P &= \mathrm{softmax}_{\mathrm{rows}}(S) \;\in\; \mathrm{M}_{n\times n}(\mathbb{R}), \\ O &= PV \;\in\; \mathrm{M}_{n\times d_v}(\mathbb{R}), \end{aligned}$$

requires IO complexity $\Theta(nd + n^2)$ where $d := d_k = d_v$.

Proof. Each step reads its inputs from HBM and writes its output to HBM (since S and P are $n \times n$ and exceed SRAM capacity M for large n):

Step	Operation	HBM reads	HBM writes
1	$S = QK^\top/\sqrt{d_k}$	$2nd$	n^2
2	$P = \mathrm{softmax}_{\mathrm{rows}}(S)$	n^2	n^2
3	$O = PV$	$n^2 + nd$	nd
	Total	$2n^2 + 3nd$	$2n^2 + nd$

Grand total: $4n^2 + 4nd = \Theta(n^2 + nd) = \Theta(nd + n^2)$. □

23. The FlashAttention Algorithm

The key insight is that the attention accumulator $(m, \ell, \tilde{o})$ from Section 20 is a commutative monoid element that can be updated *one tile at a time* entirely within SRAM, so S and P never need to be stored in HBM.

Tile parameters. Choose block sizes $B_r, B_c \in \mathbb{N}$ such that one row block of Q, one column block of K, one column block of V, and the running accumulator all fit in SRAM simultaneously:

$$B_r d_k + B_c(d_k + d_v) \;\le\; M.$$

The canonical choice is $B_c = \lfloor M/(4d) \rfloor$ and $B_r = \min\big(\lfloor M/(4d) \rfloor, d\big)$ (with $d = d_k = d_v$ for simplicity). Partition:

$$Q = \begin{pmatrix} Q_1 \\ \vdots \\ Q_{T_r} \end{pmatrix}, \qquad K = \begin{pmatrix} K_1 \\ \vdots \\ K_{T_c} \end{pmatrix}, \qquad V = \begin{pmatrix} V_1 \\ \vdots \\ V_{T_c} \end{pmatrix},$$

where $Q_i \in \mathrm{M}_{B_r\times d_k}(\mathbb{R})$, $K_j, V_j \in \mathrm{M}_{B_c\times d_k}(\mathbb{R})$ (resp. $\mathrm{M}_{B_c\times d_v}(\mathbb{R})$), and $T_r = \lceil n/B_r \rceil$, $T_c = \lceil n/B_c \rceil$.

Row-wise decomposition. By Definition 6.2, output row i of O is $O_{i,\cdot} = \mathrm{Att}(Q_{i,\cdot}, K, V) = \mathrm{softmax}(s_i)\, V$ where $s_i = Q_{i,\cdot} K^\top / \sqrt{d_k} \in \mathbb{R}^n$. By Section 20, this equals $\tilde{o}(\{1,\dots,n\})/\ell(s_i) = o(\{1,\dots,n\})$, computable as the fold

$$\alpha(\{1,\dots,n\}) \;=\; \bigoplus_{j=1}^{T_c} \alpha(A_j^{(i)}), \qquad A_j^{(i)} := \{(j-1)B_c + 1, \dots, jB_c\},$$

where $\alpha(A_j^{(i)}) = (m(s_{i,A_j}),\, \ell(s_{i,A_j}),\, \tilde{o}(A_j^{(i)}))$ depends only on $Q_{i,\cdot}$, K_j, V_j — all of which fit in SRAM.

Vectorisation over a Q block. FlashAttention processes all B_r rows of a Q block Q_i simultaneously. For block pair (i, j), define:

$$\begin{aligned}
S_{ij} &:= Q_i K_j^\top / \sqrt{d_k} \;\in\; \mathrm{M}_{B_r \times B_c}(\mathbb{R}), \\
\mathbf{m}_{ij} &:= \mathrm{rowmax}(S_{ij}) \;\in\; \mathbb{R}^{B_r}, \qquad \text{(row-wise maximum)} \\
\tilde{P}_{ij} &:= \exp\!\big(S_{ij} - \mathbf{m}_{ij} \mathbf{1}_{B_c}^\top\big) \;\in\; \mathrm{M}_{B_r \times B_c}(\mathbb{R}), \qquad \text{(row-wise shift by } \mathbf{m}_{ij}) \\
\boldsymbol{\ell}_{ij} &:= \tilde{P}_{ij}\, \mathbf{1}_{B_c} \;\in\; \mathbb{R}^{B_r}, \qquad \text{(row sums)} \\
\tilde{O}_{ij} &:= \tilde{P}_{ij} V_j \;\in\; \mathrm{M}_{B_r \times d_v}(\mathbb{R}). \qquad \text{(unnormalised partial output)}
\end{aligned}$$

The running accumulator for Q block i is the triple $(\mathbf{m}_i, \boldsymbol{\ell}_i, \tilde{O}_i) \in \mathbb{R}^{B_r} \times \mathbb{R}^{B_r} \times \mathrm{M}_{B_r \times d_v}(\mathbb{R})$, initialised to $(-\infty\, \mathbf{1}_{B_r},\, \mathbf{0}_{B_r},\, \mathbf{0})$. Each inner step j is the *row-wise* application of the merge from Proposition 20.2:

$$\begin{aligned}
\mathbf{m}_i^{\mathrm{new}} &:= \max(\mathbf{m}_i,\, \mathbf{m}_{ij}), \qquad \text{(elementwise)} \\
\boldsymbol{\ell}_i^{\mathrm{new}} &:= e^{\mathbf{m}_i - \mathbf{m}_i^{\mathrm{new}}} \odot \boldsymbol{\ell}_i \;+\; e^{\mathbf{m}_{ij} - \mathbf{m}_i^{\mathrm{new}}} \odot \boldsymbol{\ell}_{ij}, \\
\tilde{O}_i^{\mathrm{new}} &:= \mathrm{diag}\big(e^{\mathbf{m}_i - \mathbf{m}_i^{\mathrm{new}}}\big)\, \tilde{O}_i \;+\; \mathrm{diag}\big(e^{\mathbf{m}_{ij} - \mathbf{m}_i^{\mathrm{new}}}\big)\, \tilde{O}_{ij},
\end{aligned}$$

where $\odot$ denotes elementwise (Hadamard) product and $e^{\mathbf{v}}$ the elementwise exponential of a vector $\mathbf{v}$. After the inner loop: $O_i := \mathrm{diag}(\boldsymbol{\ell}_i)^{-1}\, \tilde{O}_i \in \mathrm{M}_{B_r \times d_v}(\mathbb{R})$.

Algorithm 23.1 (FlashAttention forward pass)**. Input**: $Q \in \mathrm{M}_{n \times d_k}(\mathbb{R})$, $K, V \in \mathrm{M}_{n \times d_v}(\mathbb{R})$; SRAM capacity M.

Output: $O = \mathrm{Att}(Q, K, V) \in \mathrm{M}_{n \times d_v}(\mathbb{R})$.

(1) Set $B_c \leftarrow \lfloor M/(4d) \rfloor$, $B_r \leftarrow \min(B_c, d)$, $T_r \leftarrow \lceil n/B_r \rceil$, $T_c \leftarrow \lceil n/B_c \rceil$.
(2) Initialise $O \leftarrow \mathbf{0} \in \mathrm{M}_{n \times d_v}(\mathbb{R})$ in HBM.
(3) **For** $i = 1, \dots, T_r$:
  (a) Load Q_i from HBM to SRAM. Initialise $\mathbf{m}_i \leftarrow -\infty\, \mathbf{1}$, $\boldsymbol{\ell}_i \leftarrow \mathbf{0}$, $\tilde{O}_i \leftarrow \mathbf{0}$ in SRAM.
  (b) **For** $j = 1, \dots, T_c$:
    (i) Load K_j, V_j from HBM to SRAM.
    (ii) Compute $S_{ij}, \mathbf{m}_{ij}, \tilde{P}_{ij}, \boldsymbol{\ell}_{ij}, \tilde{O}_{ij}$ in SRAM (no HBM writes).
    (iii) Update $(\mathbf{m}_i, \boldsymbol{\ell}_i, \tilde{O}_i)$ by the merge formulas above.
  (c) Write $O_i = \mathrm{diag}(\boldsymbol{\ell}_i)^{-1} \tilde{O}_i$ to HBM.

Proposition 23.2 (Correctness)**.** *Algorithm 23.1 computes $O = \mathrm{Att}(Q, K, V)$ exactly.*

Proof. Row i of O_i is the normalised output of the accumulator fold $\bigoplus_{j=1}^{T_c} \alpha(A_j^{(i)})$. By Proposition 20.2 (iterated), this equals $\alpha(\{1,\dots,n\})$, whose normalised output component is $o(\{1,\dots,n\}) = \mathrm{softmax}(s_i)\, V = O_{i,\cdot}$. □

24. IO Complexity of FlashAttention

Theorem 24.1 (FlashAttention IO complexity)**.** *Algorithm 23.1 has IO complexity* $\Theta(n^2d^2/M)$ *with the tile sizes of Step 1, where* $d = d_k = d_v$.

Proof. Count HBM accesses in the double loop:

- Q: block Q_i is loaded once per outer iteration, $B_r d$ elements each, over T_r iterations: total $T_r B_r d = nd$.
- K, V: block K_j (resp. V_j) is loaded once per inner iteration. There are $T_r \times T_c$ inner iterations: total $T_r T_c B_c d$ reads for each of K and V, i.e. $T_r nd$ reads for K and $T_r nd$ for V.
- O: block O_i is written once per outer iteration: nd writes.
- S_{ij}, P_{ij}: computed and consumed entirely in SRAM; **zero** HBM accesses.

Total: $nd + 2T_r nd + nd = (2T_r + 2)nd = \Theta(T_r nd)$.

With $B_r = \min(\lfloor M/(4d) \rfloor, d)$:

- If $M \leq 4d^2$ (typical for large d): $B_r = M/(4d)$, so $T_r = \lceil n/B_r \rceil = \Theta(4nd/M)$, giving IO $= \Theta(nd \cdot 4nd/M) = \Theta(n^2d^2/M)$.
- If $M > 4d^2$: $B_r = d$, so $T_r = \Theta(n/d)$, giving IO $= \Theta(nd \cdot n/d) = \Theta(n^2)$.

In both cases the bound is $O(n^2d^2/M)$. □

Corollary 24.2 (IO improvement over standard attention)**.** *Standard attention requires* $\Theta(nd+n^2)$ *IO (Proposition 22.1); FlashAttention requires* $O(n^2d^2/M)$*. The ratio is*

$$\frac{n^2d^2/M}{n^2} = \frac{d^2}{M}.$$

Since $M \gg d^2$ *in practice (e.g.* $M \approx 50000$ *float32 elements per SM on an A100, while* $d = 64$ *gives* $d^2 = 4096$*), FlashAttention performs* $\Theta(M/d^2)$ *fewer HBM accesses than standard attention, reducing the dominant memory bottleneck by more than an order of magnitude.*

Remark 24.3 (FLOPs are unchanged)**.** Both algorithms perform $\Theta(n^2d)$ floating-point operations (two matrix products of size $n \times d$ and $n \times n$, plus the row-wise softmax). FlashAttention does not reduce arithmetic work; it reduces *memory traffic*. The performance gain is entirely from eliminating the $\Theta(n^2)$ HBM reads and writes of S and P.

25. What Becomes Linear, and What Does Not

The phrase "reduce memory usage from quadratic to linear in sequence length" in [4] is easy to misread. It does not mean that exact dense attention has become a linear-time algorithm, nor that arbitrarily long context windows are suddenly free. It means a specific thing: the $n \times n$ score and probability matrices no longer have to be *materialised and stored*.

Definition 25.1 (Peak auxiliary memory)**.** Fix $Q, K, V \in \mathrm{M}_{n\times d}(\mathbb{R})$ and output $O \in \mathrm{M}_{n\times d}(\mathbb{R})$. The *peak auxiliary memory* of an attention algorithm is the maximum number of additional floating-point elements resident in HBM during the computation, excluding the input matrices and the final output. For training, we include quantities saved for the backward pass.

Proposition 25.2 (Standard attention has quadratic auxiliary memory)**.** *The naïve realisation*

$$S = QK^\top/\sqrt{d}, \qquad P = \mathrm{softmax}(S), \qquad O = PV$$

has peak auxiliary memory $\Theta(n^2)$.

Proof. The matrices S and P both have shape $n \times n$. A standard implementation materialises at least one of them in HBM, and often both, because softmax consumes S and the final matrix product consumes P. Thus the auxiliary footprint is bounded below and above by constant multiples of n^2 elements, ignoring the lower-order $O(nd)$ input/output terms. □

Proposition 25.3 (FlashAttention has linear auxiliary memory)**.** *Algorithm 23.1 has peak auxiliary HBM memory $\Theta(n)$ for the saved logsumexp statistic in the single-head case, and $\Theta(nd)$ if one counts input/output-shaped workspaces. In either convention it is linear in n for fixed head dimension d.*

Proof. The tiles S_{ij} and $\tilde{P}_{ij}$ are computed on chip and never written to HBM. The forward pass writes the output O and, for training, one scalar L_i per query row (Definition 28.1). The remaining live quantities are tile-local accumulators in registers/shared memory and input/output-shaped buffers. Therefore no $n \times n$ HBM object is required. □

Remark 25.4 (The word "linear" has a scope)**.** Proposition 25.3 is the precise sense in which memory becomes linear. It is a statement about *peak storage* of intermediate attention matrices, not about the number of pairwise interactions. Every query still attends to every key in dense attention.

Proposition 25.5 (Exact dense attention remains quadratic in work)**.** *For generic dense attention, computing $O = \mathrm{softmax}(QK^\top/\sqrt{d})V$ requires evaluating $\Theta(n^2)$ query-key interactions and performs $\Theta(n^2 d)$ arithmetic operations.*

Proof. The score matrix has one entry $S_{ij} = \langle Q_i, K_j\rangle/\sqrt{d}$ for every ordered pair $(i, j) \in \{1, \dots, n\}^2$. Each score is a length-d dot product, hence $\Theta(d)$ arithmetic. Multiplying the resulting attention weights by V is another dense weighted sum over the same n^2 row-column pairs, again with d output components. FlashAttention changes the order and location of these computations; it does not remove the pair set. □

Remark 25.6 (IO is also not literally linear)**.** Theorem 24.1 gives FlashAttention IO complexity $\Theta(n^2 d^2/M)$ in the two-level memory model, not $\Theta(n)$. The improvement is that S and P are not written to and read from HBM as full $n \times n$ matrices. The kernel still streams through the quadratically many query-key tiles.

Remark 25.7 (What limits context length in practice)**.** FlashAttention and FlashAttention-2 enable *larger* context windows by removing the quadratic activation-memory wall and by making the remaining quadratic work run closer to the machine's efficient matmul path. They do not remove all context-length limits. In training and prompt prefill, the dominant exact-attention cost is still the $\Theta(n^2 d)$ pairwise work of Proposition 25.5. In autoregressive decoding with a KV cache, generating one new token attends to the existing n cached keys and values, so the per-token attention work is $\Theta(nd)$ and the KV-cache memory per layer is $\Theta(nhd)$ for h heads; generating n tokens still accumulates a quadratic total attention cost. Outside the kernel itself, usable context is also limited by GPU memory capacity, memory bandwidth, batch size, number of layers, positional encoding/training length, and whether the model has learned to use information at that distance.

Thus the honest answer to "what keeps us from larger and larger context windows?" is: FlashAttention removes the need to store the full attention matrix, but exact dense attention still asks the model to process all relevant query-key pairs. The wall moves from quadratic *storage* toward quadratic *compute*, linear KV-cache storage, bandwidth, and model-quality limits.

26. FlashAttention-2: Reducing Non-Matmul FLOPs

Theorem 24.1 bounds one axis of performance: HBM traffic. A second, independent axis is *which* FLOPs are spent, since GPUs devote specialised silicon to matrix multiplication.

Remark 26.1 (Matmul vs. non-matmul FLOPs)**.** On an A100 GPU, Tensor Cores deliver a peak throughput of 312 TFLOPs/s for FP16/BF16 matrix multiply, versus only 19.5 TFLOPs/s for non-matmul FP32 arithmetic [4] — a single non-matmul FLOP costs roughly 16× a matmul FLOP. In Algorithm 23.1, the matrix products $S_{ij} = Q_i K_j^\top/\sqrt{d_k}$ and $\tilde{P}_{ij}V_j$ run on Tensor Cores; the row max, the exponential, and every division by ℓ do not. Minimising the count of the latter is therefore a distinct design goal from minimising HBM traffic.

Proposition 26.2 (Delayed normalisation)**.** *Algorithm 23.1 performs $\Theta(nd_v)$ elementwise divisions in total. The* eagerly normalised *variant — which maintains the normalised $O_i^{(j)} := \mathrm{diag}(\boldsymbol{\ell}_i^{(j)})^{-1}\tilde{O}_i^{(j)}$ after every inner-loop step j, rather than only after the loop, as in the two-block online-softmax computation of Section 19 — performs $\Theta(T_c \cdot nd_v)$ divisions, a factor of $T_c = \lceil n/B_c \rceil$ more.*

Proof. Algorithm 23.1 divides exactly once per row block, after the inner loop: $O_i := \mathrm{diag}(\boldsymbol{\ell}_i)^{-1}\tilde{O}_i$, a $B_r \times d_v$ elementwise division. Over T_r row blocks this is $T_r B_r d_v = nd_v$ divisions total.

The eager variant instead renormalises at every inner-loop step, exactly as $O^{(2)} = \mathrm{diag}(\ell^{(1)}/\ell^{(2)})^{-1}O^{(1)} + \mathrm{diag}(\ell^{(2)})^{-1}\tilde{P}^{(2)}V^{(2)}$ renormalises *both* terms of the running output at each of the two blocks [4]: one $B_r \times d_v$ division per (i, j) pair. Over $T_r T_c$ pairs this is $T_r T_c B_r d_v = T_c \cdot nd_v$ divisions total. The ratio of the two totals is T_c. □

Remark 26.3 (Delayed normalisation was already built in)**.** Algorithm 23.1 was derived directly from the attention accumulator (Definition 20.1), whose whole purpose is to carry the *unnormalised* partial output $\tilde{o}(A)$ and defer division by ℓ to the very end (Definition 20.1: $o(A) := \tilde{o}(A)/\ell(A)$). Proposition 26.2 is therefore not an optimisation applied on top of Section 23's algorithm; it is a consequence of building the algorithm from the monoid in the first place. This is the tweak [4] makes to the algorithm of [3] to obtain FlashAttention-2.

Remark 26.4 (One statistic instead of two)**.** A second FLOP-reduction tweak of [4]: rather than carrying the pair $(\mathbf{m}_i, \boldsymbol{\ell}_i)$ forward for the backward pass, only the logsumexp $L_i = m_i + \ln \ell_i$ (Definition 28.1) need be written to HBM at the end of each row block's inner loop — half the auxiliary memory traffic, and, by Proposition 28.2, sufficient to reconstruct P exactly during the backward pass (Section 29).

Proposition 26.5 (Causal tile skipping)**.** *Under the causal mask (Definition 13.1), fix row block i (global rows $(i-1)B_r + 1, \ldots, iB_r$) and column block j (global columns $(j-1)B_c + 1, \ldots, jB_c$) in Algorithm 23.1.*

(i) If $(j-1)B_c + 1 > iB_r$ — the smallest column index in block j exceeds the largest row index in block i — every entry of S_{ij} is masked, $\tilde{P}_{ij} = 0$, and the inner-loop step for (i, j) may be skipped entirely (no load of K_j, V_j, no computation).

(ii) Taking $B_r = B_c$, at most one column block per row block is only partially *masked (straddles the diagonal); every other needed block is entirely unmasked.*

Proof. (i) By Definition 13.1, $S_{i'j'}$ is masked iff $j' > i'$. The hypothesis $(j-1)B_c + 1 > iB_r$ says the smallest j' in block j already exceeds the largest i' in block i, so $j' > i'$ for every pair (i', j') in the block.

(ii) With $B_r = B_c =: B$, row block i's rows and column block i's columns cover the same index range $((i-1)B + 1, \ldots, iB)$, so the diagonal entries $S_{i'i'}$ all lie in block (i, i): it is the unique block that is neither fully masked by (i) (it contains unmasked diagonal entries) nor fully unmasked (it contains masked entries with $j' > i'$ within the block). Every column block $j < i$ is fully unmasked; every $j > i$ is skipped by (i). □

Remark 26.6 (Measured speedup)**.** By Proposition 26.5(i), roughly half of all $T_r T_c$ block pairs are skipped for large n (those with $j > i$), reducing both the matmul FLOPs and the HBM traffic of Theorem 24.1 by close to a factor of two. [4] report a 1.7–1.8× wall-clock speedup from causal masking in practice, short of the factor-of-two ideal because the unmasked row blocks (small i) finish their truncated inner loop faster than the fully-masked-free row blocks (large i), leaving some thread blocks idle while others still run — a load-imbalance cost that a purely FLOP/IO count does not capture.

27. Gradients of Scaled Dot-Product Attention

Training requires the gradients of a scalar loss through the attention map. Fix a smooth loss $\phi : \mathrm{M}_{n\times d_v}(\mathbb{R}) \to \mathbb{R}$ of the output $O = \mathrm{Att}(Q, K, V)$ and write, for any intermediate quantity X,

$$\mathrm{d}X \;:=\; \frac{\partial \phi}{\partial X},$$

a matrix of the same shape as X (the "upstream gradient" convention of reverse-mode automatic differentiation). The backward pass receives $\mathrm{d}O \in \mathrm{M}_{n\times d_v}(\mathbb{R})$ and must produce $\mathrm{d}Q, \mathrm{d}K \in \mathrm{M}_{n\times d_k}(\mathbb{R})$ and $\mathrm{d}V \in \mathrm{M}_{n\times d_v}(\mathbb{R})$.

Proposition 27.1 (Gradients through $O = PV$)**.**

$$\mathrm{d}V \;=\; P^\top \mathrm{d}O \;\in\; \mathrm{M}_{n\times d_v}(\mathbb{R}), \qquad \mathrm{d}P \;=\; \mathrm{d}O\, V^\top \;\in\; \mathrm{M}_{n\times n}(\mathbb{R}).$$

Proof. Entrywise, $O_{ia} = \sum_j P_{ij} V_{ja}$. By the chain rule,

$$\mathrm{d}V_{ja} \;=\; \sum_i \frac{\partial \phi}{\partial O_{ia}} \frac{\partial O_{ia}}{\partial V_{ja}} \;=\; \sum_i \mathrm{d}O_{ia}\, P_{ij} \;=\; (P^\top \mathrm{d}O)_{ja},$$

$$\mathrm{d}P_{ij} \;=\; \sum_a \mathrm{d}O_{ia}\, V_{ja} \;=\; \langle \mathrm{d}O_{i,\cdot},\, V_{j,\cdot}\rangle \;=\; (\mathrm{d}O\, V^\top)_{ij}. \qquad \square$$

Proposition 27.2 (Gradient through the row-wise softmax)**.** *With $P_i = \mathrm{softmax}(S_i)$ row-wise, define the row scalars*

$$D_i \;:=\; \sum_{k=1}^{n} P_{ik}\, \mathrm{d}P_{ik} \;\in\; \mathbb{R}.$$

Then

$$\mathrm{d}S_{ij} \;=\; P_{ij}\big(\mathrm{d}P_{ij} - D_i\big).$$

Proof. The Jacobian of the softmax (Remark 6.4) is $J\,\mathrm{softmax}(S_i) = \mathrm{diag}(p_i) - p_i p_i^\top$, which is symmetric, so reverse-mode application coincides with forward multiplication:

$$\mathrm{d}S_{i,\cdot} \;=\; \mathrm{d}P_{i,\cdot}\,\big(\mathrm{diag}(p_i) - p_i p_i^\top\big).$$

Entry j:

$$\mathrm{d}S_{ij} \;=\; \mathrm{d}P_{ij} P_{ij} - \Big(\sum_k \mathrm{d}P_{ik} P_{ik}\Big) P_{ij} \;=\; P_{ij}\big(\mathrm{d}P_{ij} - D_i\big). \qquad \square$$

Proposition 27.3 (The row-dot identity)**.** *D_i requires neither P nor $\mathrm{d}P$:*

$$D_i \;=\; \langle \mathrm{d}O_{i,\cdot},\, O_{i,\cdot}\rangle, \qquad \textit{i.e.} \qquad D \;=\; \mathrm{rowsum}(\mathrm{d}O \odot O) \;\in\; \mathbb{R}^n.$$

Proof. Substitute $\mathrm{d}P_{ik} = \langle \mathrm{d}O_{i,\cdot}, V_{k,\cdot}\rangle$ (Proposition 27.1) and exchange the sums:

$$D_i \;=\; \sum_k P_{ik} \langle \mathrm{d}O_{i,\cdot}, V_{k,\cdot}\rangle \;=\; \Big\langle \mathrm{d}O_{i,\cdot}, \sum_k P_{ik} V_{k,\cdot} \Big\rangle \;=\; \langle \mathrm{d}O_{i,\cdot}, O_{i,\cdot}\rangle. \qquad \square$$

This identity is what makes a memory-light backward pass possible: the $n \times n$ matrix $\mathrm{d}P$ never needs to be materialised, because its only global use — the row sums D_i — is computable from $\mathrm{d}O$ and O alone, in $\Theta(nd)$ work and memory.

Proposition 27.4 (Gradients through $S = QK^\top/\sqrt{d_k}$)**.**

$$\mathrm{d}Q \;=\; \frac{1}{\sqrt{d_k}}\, \mathrm{d}S\, K, \qquad \mathrm{d}K \;=\; \frac{1}{\sqrt{d_k}}\, \mathrm{d}S^\top Q.$$

Proof. $S_{ij} = \langle Q_{i,\cdot}, K_{j,\cdot}\rangle/\sqrt{d_k}$, so $\partial S_{ij}/\partial Q_{ib} = K_{jb}/\sqrt{d_k}$ and $\partial S_{ij}/\partial K_{jb} = Q_{ib}/\sqrt{d_k}$. Chain rule:

$$\mathrm{d}Q_{ib} = \frac{1}{\sqrt{d_k}} \sum_j \mathrm{d}S_{ij} K_{jb}, \qquad \mathrm{d}K_{jb} = \frac{1}{\sqrt{d_k}} \sum_i \mathrm{d}S_{ij} Q_{ib}. \qquad \square$$

Remark 27.5 (Causal masking)**.** With the causal mask (Definition 13.1), $P_{ij} = 0$ for $j > i$, and by Proposition 27.2 also $\mathrm{d}S_{ij} = P_{ij}(\cdots) = 0$ there. Every sum above therefore restricts to $j \le i$: masked entries contribute nothing to any gradient.

28. Recomputation and the Logsumexp Statistic

The gradient formulas of Section 27 reference P, which FlashAttention deliberately never stored. The remedy [3] is *recomputation*: the forward pass saves one scalar per row from which any tile of P can be regenerated in SRAM.

Definition 28.1 (Logsumexp statistic)**.** For row i with score row $s_i = S_{i,\cdot} \in \mathbb{R}^n$ and safe-softmax statistics $m(s_i)$, $\ell(s_i)$ (the accumulator of Section 20), define

$$L_i \;:=\; m(s_i) + \ln \ell(s_i) \;=\; \ln \sum_{j=1}^{n} e^{S_{ij}} \;=\; \mathrm{lse}(s_i).$$

Proposition 28.2 (Tile-wise recomputation of P)**.**

$$P_{ij} \;=\; e^{S_{ij} - L_i} \qquad \textit{for all } i, j.$$

Moreover the subtraction is numerically safe: $S_{ij} - L_i \le S_{ij} - m(s_i) \le 0$, so the exponential never overflows.

Proof. $e^{S_{ij}-L_i} = e^{S_{ij}-m(s_i)-\ln \ell(s_i)} = e^{S_{ij}-m(s_i)}/\ell(s_i) = P_{ij}$, the safe-softmax output. For the safety bound: the sum $\ell(s_i) = \sum_k e^{S_{ik}-m(s_i)}$ contains the term $k = j$, so $\ell(s_i) \ge e^{S_{ij}-m(s_i)}$; taking logarithms, $L_i = m(s_i) + \ln \ell(s_i) \ge S_{ij}$. $\square$

Remark 28.3 (One scalar instead of two)**.** Storing L_i replaces the pair $(m(s_i), \ell(s_i))$: n scalars total, versus the n^2 entries of P. Any $B_r \times B_c$ tile of P is then recomputable from Q_i, K_j, L resident in SRAM as $P_{ij} = \exp(Q_i K_j^\top/\sqrt{d_k} - L_i \mathbf{1}^\top)$.

Remark 28.4 (No merge monoid in the backward pass)**.** The forward pass needed the online merge (Section 18) because m and ℓ are *nonlinear* statistics of a growing key set. In the backward pass those statistics are already global — L_i is fixed — and every gradient formula is a plain *linear* sum over j (or i), which decomposes over tiles exactly. Tiled backward accumulation is therefore ordinary addition, with no rescaling and no maximum tracking.

29. The FlashAttention Backward Pass

Assemble the pieces: the forward pass leaves O and L in HBM (S, P discarded); the backward pass recomputes score and weight tiles in SRAM and accumulates the linear sums of Section 27. The algorithm below organises the work as two independent tiled passes — one parallel over row blocks producing $\mathrm{d}Q$, one parallel over column blocks producing $\mathrm{d}K, \mathrm{d}V$ — so each output block has a single writer. ([3], Appendix B, presents the same row-recompute/column-recompute equations as a single sequential sweep, outer loop over column blocks j, updating $\mathrm{d}Q_i$ in HBM at every inner step. [4] parallelises that sweep across thread blocks, one per column block j, using atomic adds to accumulate into $\mathrm{d}Q$ in HBM — see the remark on backward-pass parallelism in Section 30. The two-pass split used here instead trades one extra recomputation sweep for write locality: no atomics, no inter-block synchronisation at all.)

Algorithm 29.1 (FlashAttention backward pass)**. Input**: $Q, K \in \mathrm{M}_{n\times d_k}(\mathbb{R}), V, O, \mathrm{d}O \in \mathrm{M}_{n\times d_v}(\mathbb{R})$, $L \in \mathbb{R}^n$; tile sizes B_r, B_c as in Algorithm 23.1.

Output: $\mathrm{d}Q$, $\mathrm{d}K$, $\mathrm{d}V$.

(1) **Preprocess**: $D \leftarrow \mathrm{rowsum}(\mathrm{d}O \odot O) \in \mathbb{R}^n$ (Proposition 27.3).

(2) **Pass 1 ($\mathrm{d}Q$), for** each row block $i = 1, \dots, T_r$:

(a) Load $Q_i, \mathrm{d}O_i, L_i, D_i$ into SRAM; $\mathrm{d}Q_i \leftarrow \mathbf{0}$.

(b) **For** $j = 1, \dots, T_c$: load K_j, V_j; recompute in SRAM

$$S_{ij} = Q_i K_j^\top / \sqrt{d_k}, \qquad P_{ij} = \exp(S_{ij} - L_i \mathbf{1}^\top),$$

$$\mathrm{d}P_{ij} = \mathrm{d}O_i V_j^\top, \qquad \mathrm{d}S_{ij} = P_{ij} \odot (\mathrm{d}P_{ij} - D_i \mathbf{1}^\top),$$

$$\mathrm{d}Q_i \;\mathrel{+}=\; \mathrm{d}S_{ij}\, K_j / \sqrt{d_k}.$$

(c) Write $\mathrm{d}Q_i$ to HBM.

(3) **Pass 2 ($\mathrm{d}K, \mathrm{d}V$), for** each column block $j = 1, \dots, T_c$:

(a) Load K_j, V_j into SRAM; $\mathrm{d}K_j, \mathrm{d}V_j \leftarrow \mathbf{0}$.

(b) **For** $i = 1, \dots, T_r$: load $Q_i, \mathrm{d}O_i, L_i, D_i$; recompute $S_{ij}, P_{ij}, \mathrm{d}P_{ij}, \mathrm{d}S_{ij}$ as in Pass 1, then

$$\mathrm{d}V_j \;\mathrel{+}=\; P_{ij}^\top\, \mathrm{d}O_i, \qquad \mathrm{d}K_j \;\mathrel{+}=\; \mathrm{d}S_{ij}^\top\, Q_i / \sqrt{d_k}.$$

(c) Write $\mathrm{d}K_j, \mathrm{d}V_j$ to HBM.

Proposition 29.2 (Correctness)**.** *Algorithm 29.1 computes the exact gradients of Section 27.*

Proof. By Proposition 28.2 each recomputed tile P_{ij} is exact, hence so are $\mathrm{d}P_{ij}$ and $\mathrm{d}S_{ij}$ (Propositions 27.1, 27.2, 27.3). The remaining sums $\mathrm{d}Q_i = \sum_j \mathrm{d}S_{ij} K_j / \sqrt{d_k}$, $\mathrm{d}V_j = \sum_i P_{ij}^\top \mathrm{d}O_i$, and $\mathrm{d}K_j = \sum_i \mathrm{d}S_{ij}^\top Q_i / \sqrt{d_k}$ are finite sums over the tile partition (Proposition 27.4), and tile-wise accumulation computes them term by term. □

Remark 29.3 (Causal tiling)**.** Under the causal mask, Pass 1 for row block i needs only column tiles with first key index $\le$ the block's last row (later tiles are entirely masked), and Pass 2 for column block j needs only row tiles with last row index $\ge$ the block's first key — each pass skips roughly half its inner iterations, mirroring the forward tile skipping.

Remark 29.4 (IO complexity)**.** Each pass has the same loop structure as the forward pass — every inner iteration moves $\Theta(Bd)$ elements and there are $T_r T_c$ inner iterations — so the backward IO complexity is again $\Theta(n^2 d^2 / M)$ (Theorem 24.1), with S, P, $\mathrm{d}S$, $\mathrm{d}P$ never touching HBM. The extra inputs O, $\mathrm{d}O$, L, D contribute only $\Theta(nd)$.

30. Parallelism and Work Partitioning

Theorem 24.1 and Proposition 26.2 bound work *per SM*; a third, independent axis is how that work is spread across the GPU's many SMs and, within one thread block, across its warps.

Remark 30.1 (Thread-block parallelism over row blocks)**.** [3] schedules one thread block per (batch, head) pair: with batch size β and h heads, βh thread blocks in total, one per call to Att from Definition 6.2. This is efficient when βh is at least the GPU's SM count (108 on an A100), but under-uses the device when βh is small — exactly the long-sequence, small-batch regime attention is applied in at inference time.

The outer loop of Algorithm 23.1 (over row blocks $i = 1, \dots, T_r$) has no data dependency between iterations: row block i reads only Q_i, K, V and writes only O_i, L_i. [4] (crediting the loop-order swap and this parallelisation strategy to Tillet's Triton implementation [5]) therefore schedules $\beta h T_r$ thread blocks — one per row block, per head, per batch element — recovering full SM occupancy even when βh is small.

Remark 30.2 (The loop order that enables it)**.** Algorithm 23.1, as stated in Section 23, already has the outer loop over row blocks i and the inner loop over column blocks j. This is what makes Remark 30.1 possible: each thread block owns one triple $(\mathbf{m}_i, \boldsymbol{\ell}_i, \tilde{O}_i)$ in registers/SRAM for the entire inner loop and writes O_i exactly once, with zero communication to any other thread block. Swapping the loop order (outer over column blocks j, inner over row blocks i) — the order used for the backward pass in Section 29 because $\mathrm{d}Q$ accumulates across column blocks — would instead force every column-block iteration to touch every row's accumulator, either serialising all T_c column iterations within one thread block per row (as in Remark 30.1's original scheme) or requiring cross-block synchronisation to update a shared accumulator.

Remark 30.3 (Warp-level partitioning: split-K vs. split-Q)**.** Within one thread block computing row block i against column block j, suppose there are W warps. Two ways to divide the $B_r \times B_c$ computation of S_{ij} among them:

- **Split-K** [3]: partition the B_c columns of K_j, V_j across the W warps (B_c/W columns each). Each warp computes a partial $\tilde{P}_{ij}V_j$ slice using *all* B_r rows, so the W partial $B_r \times d_v$ results must be summed across warps — a write to shared memory, a barrier, and a reduction, once per tile.
- **Split-Q** [4]: partition the B_r rows of Q_i across the W warps instead (B_r/W rows each). Each warp computes S_{ij}, $\tilde{P}_{ij}$, and $\tilde{P}_{ij}V_j$ for *only its own rows*, reading K_j, V_j from shared memory as a broadcast. No cross-warp communication is needed at all.

Taking $B_r = W$ (one row per warp) is the finest-grained instance of split-Q: each warp owns a single row's full accumulator (Definition 20.1) end to end, and the inner loop of Algorithm 23.1 runs independently per warp with zero inter-warp communication.

Remark 30.4 (Backward-pass parallelism: atomics vs. recomputation)**.** As noted in Section 29, [3]'s Appendix B backward algorithm is a single sequential sweep, outer loop over column blocks j, that reads-modifies-writes $\mathrm{d}Q_i$ in HBM at every inner step i. [4] parallelises this sweep by scheduling one thread block per column block j and using atomic adds to accumulate each block's contribution into $\mathrm{d}Q$ — the only point in either pass where different thread blocks write to overlapping output. The two-pass Algorithm 29.1 of this document avoids the atomics entirely, at the cost of a second full sweep that recomputes every S_{ij}, P_{ij} tile already recomputed in the first pass. By Remark 26.1, that extra sweep is $\Theta(n^2 d)$ additional *matmul* FLOPs (recomputing S_{ij} and $\tilde{P}_{ij}V_j$-shaped products) — cheap, by the same hardware asymmetry that motivated Proposition 26.2 — traded against atomic memory operations and inter-block synchronisation, which are not a FLOP cost at all but a memory-contention one. Which trade wins is a hardware- and shape-dependent question outside the scope of the FLOP/IO counting used elsewhere in this document.

Remark 30.5 (Multi-query and grouped-query attention)**.** Multi-query attention [6] and grouped-query attention [7] modify Definition 10.1 by sharing key/value weight matrices across heads: fix a group size $g \mid h$, and replace W_ℓ^K, W_ℓ^V in the definition of K_ℓ, V_ℓ by $W_{\lceil \ell/g \rceil}^K, W_{\lceil \ell/g \rceil}^V$, so only h/g distinct (K, V) pairs are computed and stored ($g = h$ is multi-query attention, the single-shared-pair case; $g = 1$ recovers Definition 10.1 exactly). This shrinks the key/value cache used at autoregressive inference time by a factor of g without changing Att's per-head computation; in the backward pass, gradients $\mathrm{d}K, \mathrm{d}V$ for the g heads sharing a (K, V) pair must be summed. We do not develop this variant further here.

31. FlashAttention-2 from First Principles

We now reinterpret the claims of [4] in the language developed above. The point is not to restate the paper's engineering narrative, but to identify the mathematical invariants that make the algorithmic changes legal. From this perspective FlashAttention-2 is not a new attention map. It is a different *representative* and a different *schedule* for computing the same smooth map Att of

Definition 6.2, together with a resource-sensitive preference order on schedules induced by the GPU memory hierarchy and by the relative cost of matmul and non-matmul operations.

Definition 31.1 (Resource-annotated realisation)**.** Fix input and output spaces X, Y over $\mathbb{R}$. A *realisation* of a map $f : X \to Y$ is a finite directed acyclic graph whose vertices are elementary operations over $\mathbb{R}$ and whose composite map is f. A *resource annotation* assigns to each vertex and edge a cost vector, e.g.

$$(\text{matmul FLOPs},\ \text{non-matmul FLOPs},\ \text{HBM reads},\ \text{HBM writes},\ \text{barriers/atomics}).$$

Two realisations are *extensionally equal* if their composites as maps $X \to Y$ are equal; they may still be different computational objects because their annotations differ.

Remark 31.2 (What FlashAttention-2 changes)**.** At Level 2, standard attention, FlashAttention, and FlashAttention-2 all realise the same map

$$(Q, K, V) \longmapsto \mathrm{softmax}(QK^\top/\sqrt{d_k})V.$$

The claims of [4] live in Definition 31.1: they replace one resource-annotated realisation by an extensionally equal one with fewer expensive non-matmul operations, more independent thread blocks, and less cross-warp communication. Correctness is a statement about equality of composites; speed is a statement about the annotation.

Proposition 31.3 (Delayed normalisation is choice of representative)**.** *For a non-empty key set A, the attention accumulator $(m(A), \ell(A), \tilde{o}(A))$ of Definition 20.1 represents the output only through the quotient*

$$o(A) = \frac{\tilde{o}(A)}{\ell(A)}.$$

Thus replacing the normalised representative $o(A)$ by the unnormalised pair $(\tilde{o}(A), \ell(A))$ throughout the inner loop preserves the represented attention output, provided the final normalisation by $\ell(A)$ is performed.

Proof. The equality is exactly the definition of $o(A)$. For a tile partition $A = A_1 \sqcup \cdots \sqcup A_t$, Proposition 20.2 computes $(m(A), \ell(A), \tilde{o}(A))$ by repeated merges. No intermediate normalised vector $o(A_1 \sqcup \cdots \sqcup A_j)$ is needed to form the next merge; the merge formulas use only m, ℓ, and $\tilde{o}$. Therefore normalising after every tile and normalising once at the end yield the same $o(A)$. □

Corollary 31.4 (The non-matmul-FLOP claim)**.** *The first algorithmic claim of* [4] *follows from Proposition 31.3: divisions by ℓ are not semantic operations of the fold, but only changes of representative from $(\tilde{o}, \ell)$ to o. Moving them to the epilogue cannot change* Att *and removes all inner-loop normalisations.*

Proposition 31.5 (Logsumexp is a sufficient statistic for the backward pass)**.** *For the backward formulas of Section 27, the single scalar*

$$L_i = \log \sum_j e^{S_{ij}}$$

is a sufficient row statistic for reconstructing every tile of the softmax matrix P.

Proof. This is Proposition 28.2: $P_{ij} = e^{S_{ij} - L_i}$. The reverse-mode formulas use P only through tile-local expressions P_{ij}, $P_{ij}\mathrm{d}O_i$, and $P_{ij}(\mathrm{d}P_{ij} - D_i)$. Once Q_i, K_j, and L_i are resident on chip, each such term is determined. Neither m_i nor ℓ_i is needed separately. □

Theorem 31.6 (FlashAttention-2 forward correctness)**.** *The FlashAttention-2 forward algorithm of* [4]*, viewed as Algorithm 23.1 with the delayed-normalisation representative of Proposition 31.3 and the stored statistic of Proposition 31.5, computes exactly the attention map* $\mathrm{Att}(Q, K, V)$ *over* $\mathbb{R}$.

Proof. Algorithm 23.1 is correct by Proposition 23.2. Delayed normalisation changes only the representative of the accumulator, not the represented output, by Proposition 31.3. Storing L is an auxiliary output for training and does not feed back into the forward output O. Therefore the composite map from (Q, K, V) to O is unchanged. □

Proposition 31.7 (Row-block parallelism is independence of product factors)**.** *Let $F_i(Q, K, V)$ denote row block O_i of the attention output produced by Algorithm 23.1. Then*

$$\mathrm{Att}(Q, K, V) = (F_1(Q, K, V), \ldots, F_{T_r}(Q, K, V))$$

as a product map into $\mathrm{M}_{B_r \times d_v}(\mathbb{R})^{T_r}$, and the maps F_i have no write-write dependency.

Proof. The i-th output block depends on Q_i and on the read-only matrices K, V. It writes only O_i and, for training, L_i. For $i \neq i'$ the write sets $\{O_i, L_i\}$ and $\{O_{i'}, L_{i'}\}$ are disjoint. The product decomposition is therefore literal: the row-block computations are the coordinate maps of Att with respect to the row-block decomposition of the codomain. □

Corollary 31.8 (The sequence-parallelism claim)**.** *Scheduling one thread block per row block, batch element, and head is a legal parallel schedule because it evaluates independent product factors of Proposition 31.7. It changes occupancy, not the mathematical map.*

Definition 31.9 (Reduction index of a tile computation)**.** For a tile product $\tilde{P}_{ij} V_j$, call an index a *reduction index* if partial results over different values of that index must be summed to obtain a single output row. In $\sum_{a \in A_j} \tilde{P}_{ia} V_a$, the key index a is a reduction index; the query row index i is not.

Proposition 31.10 (Split-Q removes cross-warp reductions)**.** *For the forward tile computation, a partition of work across warps along a non-reduction index requires no cross-warp summation of output rows. A partition along a reduction index does require such a summation unless one warp owns the whole reduction.*

Proof. If warp w owns a set of query rows, it computes complete sums $\sum_{a \in A_j} \tilde{P}_{ia} V_a$ for those rows; no other warp contributes to the same output row. This is split-Q. If instead warp w owns a proper subset $A_{j,w}$ of keys, it computes only $\sum_{a \in A_{j,w}} \tilde{P}_{ia} V_a$. The complete output row is the sum over w of these partial results, so cross-warp communication is forced by linearity of the reduction itself. This is split-K. □

Corollary 31.11 (The warp-partitioning claim)**.** *The split-Q work partition of [4] is preferable, all else equal, because it partitions a product index rather than a reduction index. Its advantage is not a new algebraic identity; it is the absence of an otherwise necessary reduction edge in the resource-annotated realisation.*

Proposition 31.12 (The causal mask is a lower set)**.** *Let $[n] \times [n]$ carry the product order and define*

$$C := \{(i, j) : j \leq i\}.$$

Then C is a lower set in the second coordinate relative to each fixed row: if $(i, j) \in C$ and $j' \leq j$, then $(i, j') \in C$. For rectangular tiles $R \times A$, a tile is entirely skipped precisely when

$$\min A > \max R.$$

Proof. The lower-set statement is immediate from $j' \leq j \leq i$. The tile $R \times A$ is entirely outside C exactly when every $(i, j) \in R \times A$ satisfies $j > i$, which is equivalent to the smallest column index exceeding the largest row index: $\min A > \max R$. □

Remark 31.13 (Load imbalance as non-uniform fibre size)**.** For square causal tiles, the number of live column tiles in row block i is approximately i. Thus the projection from live tile pairs $\{(i, j) : j \leq i\}$ to row blocks has fibres of different cardinalities. The causal speedup is therefore

bounded by the triangular count of live tiles, but the realised wall time also depends on how these unequal fibres are scheduled across SMs. This is the mathematical content behind the load-imbalance remark following Proposition 26.5.

Theorem 31.14 (Backward correctness from the differential)**.** *Any tiled backward algorithm that, for every tile (i, j), recomputes P_{ij} from Q_i, K_j, L_i and accumulates the three bilinear expressions*

$$\mathrm{d}Q_i \mathrel{+}= \mathrm{d}S_{ij}K_j/\sqrt{d_k}, \qquad \mathrm{d}K_j \mathrel{+}= \mathrm{d}S_{ij}^\top Q_i/\sqrt{d_k}, \qquad \mathrm{d}V_j \mathrel{+}= P_{ij}^\top \mathrm{d}O_i$$

with $\mathrm{d}S_{ij} = P_{ij} \odot (\mathrm{d}O_i V_j^\top - D_i \mathbf{1}^\top)$ and $D_i = \langle \mathrm{d}O_i, O_i \rangle$ computes the reverse-mode derivative of Att.

Proof. The displayed expressions are exactly the block restrictions of Propositions 27.1, 27.2, 27.3, and 27.4. By Proposition 31.5, the recomputed P_{ij} equals the corresponding tile of the full softmax matrix. Summing over all tiles is summing over a partition of the finite index set $[n] \times [n]$, so the result is the same finite sum as the untiled reverse-mode derivative. □

Remark 31.15 (Atomics are scheduling, not calculus)**.** [4] accumulates $\mathrm{d}Q$ contributions from multiple column blocks using atomics. Algorithm 29.1 instead uses a two-pass schedule with a single writer for each output block. Theorem 31.14 explains why both schedules are mathematically valid: they differ only in how the finite sums are associated and where write conflicts are resolved. The choice belongs to the resource annotation, not to the differential formula.

Proposition 31.16 (Grouped-query attention as a quotient on heads)**.** *Let $\pi : \{1, \dots, h\} \to \{1, \dots, h/g\}$ be $\pi(\ell) = \lceil \ell/g \rceil$. Grouped-query attention replaces the independent key/value head index ℓ by the quotient index $\pi(\ell)$. In the backward pass, the gradient with respect to a shared key/value head is the sum over the fibre of π:*

$$\mathrm{d}K_r^{\mathrm{shared}} = \sum_{\ell \in \pi^{-1}(r)} \mathrm{d}K_\ell, \qquad \mathrm{d}V_r^{\mathrm{shared}} = \sum_{\ell \in \pi^{-1}(r)} \mathrm{d}V_\ell.$$

Proof. The forward map duplicates the shared variables K_r, V_r into each head ℓ lying over r. The derivative of the diagonal duplication map $x \mapsto (x, \dots, x)$ is the same diagonal map, and its adjoint under the Euclidean pairing is summation over components. Hence reverse-mode differentiation sums the per-query-head gradients over each fibre. □

Remark 31.17 (What is, and is not, proved by these notes)**.** The results above prove equality of real-valued maps and identify the resource terms affected by FlashAttention-2's transformations. They do not prove a universal wall-clock speedup. Wall-clock claims depend on the Level 1 realisation: Tensor Core use, register pressure, shared-memory bank conflicts, occupancy, cache behaviour, atomics, compiler choices, and the particular GPU. Those are engineering theorems about a concrete machine, not consequences of smoothness, associativity, or finite-dimensional linear algebra alone.

References

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30*, 2017. arXiv:1706.03762.

[2] Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax. 2018. arXiv:1805.02867.

[3] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems 35*, 2022. arXiv:2205.14135.

[4] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations*, 2024. arXiv:2307.08691.

[5] Philippe Tillet, Hsiang-Tsung Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pages 10–19, 2019.

[6] Noam Shazeer. Fast transformer decoding: One write-head is all you need. 2019. arXiv:1911.02150.
[7] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. 2023. arXiv:2305.13245.

Email address: ernestyalumni@gmail.com